Examples/DeepMergingEngine.kt - Motor de Mesclagem Profunda O(N^2) Otimizado

```
 */
class DeepOrderPoolMergingEngine private constructor(private val context: Context) {

    companion object {
        private const val TAG = "DeepOrderPoolEngine"

        @Volatile
        private var INSTANCE: DeepOrderPoolMergingEngine? = null

        fun getInstance(context: Context): DeepOrderPoolMergingEngine {
            return INSTANCE ?: synchronized(this) {
                INSTANCE ?: DeepOrderPoolMergingEngine(context.applicationContext).also {
INSTANCE = it }
            }
        }
    }

    // Lista de todas as ofertas ativas escaneadas dos feeds
    private val _scannedPoolOffers = MutableStateFlow<List<ScreenPoolOffer>>(emptyList())
    val scannedPoolOffers: StateFlow<List<ScreenPoolOffer>> = _scannedPoolOffers.asStateFl
ow()

    // Melhores combos rankeados de mesclagem profunda
    private val _rankedCombos = MutableStateFlow<List<MergedDeliveryCombo>>(emptyList())
    val rankedCombos: StateFlow<List<MergedDeliveryCombo>> = _rankedCombos.asStateFlow()

    // Estat?sticas de varredura
    private val _lastScanTimestamp = MutableStateFlow(0L)
    val lastScanTimestamp: StateFlow<Long> = _lastScanTimestamp.asStateFlow()

    private val _scannedAppsCount = MutableStateFlow(0)
    val scannedAppsCount: StateFlow<Int> = _scannedAppsCount.asStateFlow()

    /**
     * Atualiza a lista de ofertas escaneadas de um app espec?fico e recalcula a matriz co
mbinat?ria.
     */
    @Synchronized
    fun ingestPoolOffers(appName: String, offers: List<ScreenPoolOffer>) {
        if (offers.isEmpty()) return

        val currentList = _scannedPoolOffers.value.toMutableList()
        // Remove ofertas anteriores deste mesmo app que tenham expirado (mais de 4 minuto
s)
        val cutoff = System.currentTimeMillis() - (4 * 60 * 1000L)
        currentList.removeAll { it.appName.equals(appName, ignoreCase = true) || it.timest
amp < cutoff }

        // Adiciona as novas filtrando dados corrompidos ou zerados
        val validOffers = offers.filter { it.value > 0.0 && it.distanceKm > 0.05 && it.dis
tanceKm.isFinite() }
        currentList.addAll(validOffers)
        _scannedPoolOffers.value = currentList.distinctBy { "${it.appName}_${it.restaurant
Name}_${it.value}" }
        _lastScanTimestamp.value = System.currentTimeMillis()
        _scannedAppsCount.value = _scannedPoolOffers.value.map { it.appName }.distinct().s
ize

        Log.i(TAG, "? Ingest?o de pool: ${validOffers.size} ofertas v?lidas de $appName. T
otal no banco: ${_scannedPoolOffers.value.size}")
        recalculateOptimalCombos()
    }

    /**
     * Recalcula os melhores combos combinando pares de ofertas e calculando lucro l?quido
 real.
     */
    fun recalculateOptimalCombos() {
        val pool = _scannedPoolOffers.value.toList()
        if (pool.size < 2) {
            _rankedCombos.value = emptyList()
            return
        }

        CoroutineScope(Dispatchers.Default).launch {
            val fuelPrice = FuelManager.currentFuelPrice.coerceAtLeast(1.0)
            val kmPerLiter = max(FuelManager.getEffectiveKmPerLiter(), 10.0)
            val costPerKm = (fuelPrice / kmPerLiter).coerceAtLeast(0.01)

            val currentLoc = LocationService.globalLocationState.value
            val combos = mutableListOf<MergedDeliveryCombo>()
```

```
            // Combina pares de ofertas (de apps diferentes ou do mesmo app se rota compat
?vel)
            for (i in 0 until pool.size) {
                for (j in (i + 1) until pool.size) {
                    val o1 = pool[i]
                    val o2 = pool[j]

                    val combo = evaluatePair(o1, o2, costPerKm, currentLoc)
                    if (combo != null && combo.slaSafetyScore >= 70 && combo.extraDeviatio
nMeters <= 2200) {
                        combos.add(combo)
                    }
                }
            }

            // Ordena priorizando Menor Desvio no mesmo corredor e Maior Lucro L?quido por
 Hora
            val sortedCombos = combos.sortedWith(
                compareByDescending<MergedDeliveryCombo> { it.isDiamondDeal }
                    .thenBy { it.extraDeviationMeters }
                    .thenBy { it.corridorAlignmentDeg }
                    .thenByDescending { it.netProfitPerHour }
                    .thenByDescending { it.slaSafetyScore }
            ).take(8)

            _rankedCombos.value = sortedCombos
            Log.i(TAG, "? Combos calculados: ${combos.size} v?lidos. Top ${sortedCombos.si
ze} exibidos no mesmo corredor.")

        // Se houver um combo diamante espetacular, avisa via ?udio/voz se n?o estiver em
alta velocidade
        val top = sortedCombos.firstOrNull()
        if (top != null && top.isDiamondDeal && !currentLoc.isSafetyLockActive) {
            val text = "Combo Diamante de Corredor detectado: ${top.primaryOffer.appName}
mais ${top.secondaryOffer.appName}. Ganho total de R$ ${String.format(Locale.US, "%.2f", t
op.totalGrossRevenue)} com lucro de R$ ${String.format(Locale.US, "%.2f", top.netProfit)}
e desvio de apenas ${top.extraDeviationMeters} metros."
            try {
                TextToSpeechManager.getInstance(context).speakVoiceAlert(text)
            } catch (_: Exception) {}

            try {
                if (OverlayWindowManager.canDrawOverlays(context)) {
                    val pricePerKm = if (top.combinedDistanceKm > 0) top.totalGrossRevenue
 / top.combinedDistanceKm else 0.0
                    OverlayWindowManager.getInstance(context).showTacticalOffer(
                        OverlayWindowManager.FloatingTacticalOffer(
                            appName = "${top.primaryOffer.appName} + ${top.secondaryOffer.
appName}",
                            packageName = top.primaryOffer.packageName,
                            value = top.totalGrossRevenue,
                            distanceKm = top.combinedDistanceKm,
                            pricePerKm = pricePerKm,
                            isGoodDeal = true,
                            dealLabel = String.format(Locale.US, "? CORREDOR +R$ %.2f (+%d
m)", top.netProfit, top.extraDeviationMeters),
                            destination = "${top.primaryOffer.dropoffAddress} ? +${top.ext
raDeviationMeters}m",
                            pickup = "${top.primaryOffer.restaurantName} & ${top.secondary
Offer.restaurantName}",
                            hasDualRoute = true,
                            dualRouteAppName = top.secondaryOffer.appName,
                            dualRouteExtraGain = top.secondaryOffer.value,
                            haversineDeviationMeters = top.extraDeviationMeters,
                            haversineExtraMinutes = top.estimatedExtraMinutes.toDouble()
                        )
                    )
                }
            } catch (_: Exception) {}
        }
    }
}

    /**
     * Avalia a compatibilidade f?sica e financeira de duas corridas simult?neas com foco
no corredor geogr?fico.
     */
    private fun evaluatePair(
        o1: ScreenPoolOffer,
        o2: ScreenPoolOffer,
        costPerKm: Double,
        currentLoc: LocationSpeedState
```

```
    ): MergedDeliveryCombo? {
        val grossRev = o1.value + o2.value

        // Dist?ncia entre coletas
        val distBetweenPickups = calculateGeodeticKm(o1.pickupLat, o1.pickupLng, o2.pickup
Lat, o2.pickupLng)
        // Dist?ncia entre entregas
        val distBetweenDropoffs = calculateGeodeticKm(o1.dropoffLat, o1.dropoffLng, o2.dro
poffLat, o2.dropoffLng)

        // Se coletas ou entregas forem excessivamente distantes (acima de 2.8 km), descar
ta
        if (distBetweenPickups > 2.8 || distBetweenDropoffs > 3.2) {
            return null
        }

        // An?lise de Vetor Direcional (Azimute de Deslocamento das duas corridas)
        val bearing1 = calculateBearing(o1.pickupLat, o1.pickupLng, o1.dropoffLat, o1.drop
offLng)
        val bearing2 = calculateBearing(o2.pickupLat, o2.pickupLng, o2.dropoffLat, o2.drop
offLng)
        val angularDiff = calculateAngularDifference(bearing1, bearing2)

        // Se as duas rotas seguem em dire??es opostas (diverg?ncia angular > 70 graus) e
a entrega n?o for hiper-pr?xima, descarta para evitar contra-fluxo
        if (angularDiff > 70f && distBetweenDropoffs > 1.2) {
            return null
        }

        // Rota combinada estimada: Coleta 1 -> Coleta 2 -> Entrega 1 -> Entrega 2
        // Dist?ncia combinada: maior perna + o desvio entre coletas e entregas com fator
de malha urbana (1.18x)
        val maxDirect = max(o1.distanceKm, o2.distanceKm)
        val extraKm = (distBetweenPickups + distBetweenDropoffs) * 1.18
        val totalKm = maxDirect + extraKm
        val deviationMeters = (extraKm * 1000).roundToInt()
        val pickupDevMeters = (distBetweenPickups * 1000).roundToInt()
        val dropoffDevMeters = (distBetweenDropoffs * 1000).roundToInt()

        // Estimativa de tempo total e extra
        val extraMinutes = ((extraKm / 28.0) * 60.0).roundToInt().coerceAtLeast(3)
        val totalMinutes = max(o1.estimatedMinutes, o2.estimatedMinutes) + extraMinutes

        // Lucro l?quido: Receita bruta menos o custo de combust?vel
        val fuelCost = totalKm * costPerKm
        val netProfit = (grossRev - fuelCost).let { if (it.isFinite()) it else 0.0 }
        val rawNetPerHour = if (totalMinutes > 0) (netProfit / totalMinutes) * 60.0 else 0
.0
        val netProfitPerHour = if (rawNetPerHour.isFinite()) rawNetPerHour.coerceAtLeast(0
.0) else 0.0

        // Score de Seguran?a SLA (Evita atraso na entrega de comida quente)
        val anglePenalty = (angularDiff / 10).roundToInt()
        val slaScore = (100 - (extraMinutes * 3) - (deviationMeters / 90) - anglePenalty).
coerceIn(10, 100)

        // Combo Diamante: mais de R$ 26,00 bruto, mais de R$ 45,00/h l?quida e desvio < 1
.000m com ?ngulo <= 45?
        val isDiamond = grossRev >= 26.0 && netProfitPerHour >= 45.0 && deviationMeters <=
 1000 && angularDiff <= 45f

        val corridorLabel = when {
            angularDiff <= 25f -> "Corredor Direcional Alinhado (${angularDiff.roundToInt(
)}?)"
            angularDiff <= 45f -> "Mesmo Corredor Vi?rio (${angularDiff.roundToInt()}?)"
            else -> "Corredor Transversal Compat?vel (${angularDiff.roundToInt()}?)"
        }

        val steps = listOf(
            "1. Coletar em ${o1.restaurantName} (${o1.appName})",
            "2. Coletar em ${o2.restaurantName} (${o2.appName}) - Desvio: +${pickupDevMete
rs}m",
            "3. Entregar em ${o1.dropoffAddress} (${o1.appName})",
            "4. Entregar em ${o2.dropoffAddress} (${o2.appName}) - Desvio: +${dropoffDevMe
ters}m"
        )

        val reason = if (isDiamond) {
            "? Combo Diamante de Corredor: $corridorLabel com ganho l?quido de R$ ${String
.format(Locale.US, "%.2f", netProfitPerHour)}/h e apenas +${deviationMeters}m de desvio."
        } else {
            "? Mesclagem de Corredor: $corridorLabel com rendimento l?quido de R$ ${String
```

```
.format(Locale.US, "%.2f", netProfitPerHour)}/h (+${extraMinutes} min)."
        }

        val driverLat = if (currentLoc.latitude != 0.0) currentLoc.latitude else -23.56168
4
        val driverLng = if (currentLoc.longitude != 0.0) currentLoc.longitude else -46.655
981
        val driverSpeed = currentLoc.calculatedSpeedKmh

        // Avalia??o de desvio geod?sico Haversine e limite din?mico de SLA
        val haversineEval = HaversineGeoDeviationCalculator.evaluateMultiPointRouteDeviati
on(
            driverLat = driverLat,
            driverLng = driverLng,
            driverSpeedKmh = driverSpeed,
            o1 = o1,
            o2 = o2,
            slaMinutes = 25
        )

        // Se o desvio Haversine violar o limite din?mico com risco de atraso (CR?TICO), d
escarta
        if (haversineEval.statusLevel == DeviationStatusLevel.CRITICAL_EXCESSIVE && !haver
sineEval.isWithinDynamicTolerance) {
            return null
        }

        return MergedDeliveryCombo(
            primaryOffer = o1,
            secondaryOffer = o2,
            totalGrossRevenue = grossRev,
            combinedDistanceKm = Math.round(totalKm * 10.0) / 10.0,
            extraDeviationMeters = haversineEval.netDeviationMeters,
            estimatedExtraMinutes = haversineEval.estimatedExtraMinutes.roundToInt(),
            fuelCostEstimated = Math.round(fuelCost * 100.0) / 100.0,
            netProfit = Math.round(netProfit * 100.0) / 100.0,
            netProfitPerHour = Math.round(netProfitPerHour * 100.0) / 100.0,
            slaSafetyScore = slaScore,
            executionSteps = steps,
            recommendationReason = reason,
            isDiamondDeal = isDiamond,
            corridorAlignmentDeg = angularDiff,
            corridorLabel = corridorLabel,
            pickupDeviationMeters = haversineEval.distancePickup1ToPickup2Meters,
            dropoffDeviationMeters = haversineEval.distanceDropoff1ToDropoff2Meters,
            haversineResult = haversineEval
        )
    }

    private fun calculateBearing(lat1: Double, lon1: Double, lat2: Double, lon2: Double):
Float {
        val phi1 = Math.toRadians(lat1)
        val phi2 = Math.toRadians(lat2)
        val deltaLambda = Math.toRadians(lon2 - lon1)

        val y = Math.sin(deltaLambda) * Math.cos(phi2)
        val x = Math.cos(phi1) * Math.sin(phi2) - Math.sin(phi1) * Math.cos(phi2) * Math.c
os(deltaLambda)
        val theta = Math.atan2(y, x)
        val bearing = (Math.toDegrees(theta) + 360.0) % 360.0
        return bearing.toFloat()
    }

    private fun calculateAngularDifference(bearing1: Float, bearing2: Float): Float {
        val diff = Math.abs(bearing1 - bearing2) % 360f
        return if (diff > 180f) 360f - diff else diff
    }

    private fun calculateGeodeticKm(lat1: Double, lon1: Double, lat2: Double, lon2: Double
): Double {
        val results = FloatArray(1)
        Location.distanceBetween(lat1, lon1, lat2, lon2, results)
        return (results[0] / 1000.0)
    }

    /**
     * Limpa ofertas antigas ou reinicia o scanner.
     */
    fun clearPool() {
        _scannedPoolOffers.value = emptyList()
        _rankedCombos.value = emptyList()
    }
```

```
}
```

```
 */
object HaversineGeoDeviationCalculator {

    private const val EARTH_RADIUS_METERS = 6371000.0

    /**
     * Calcula a dist?ncia geod?sica em metros entre dois pontos (WGS84).
     */
    fun calculateHaversineDistanceMeters(
        lat1: Double,
        lon1: Double,
        lat2: Double,
        lon2: Double
    ): Double {
        val phi1 = Math.toRadians(lat1)
        val phi2 = Math.toRadians(lat2)
        val deltaPhi = Math.toRadians(lat2 - lat1)
        val deltaLambda = Math.toRadians(lon2 - lon1)

        val a = sin(deltaPhi / 2.0) * sin(deltaPhi / 2.0) +
                cos(phi1) * cos(phi2) *
                sin(deltaLambda / 2.0) * sin(deltaLambda / 2.0)

        val c = 2.0 * atan2(sqrt(a), sqrt(1.0 - a))
        return EARTH_RADIUS_METERS * c
    }

    /**
     * Calcula o azimute (rumo inicial em graus [0..360]) entre dois pontos.
     */
    fun calculateBearing(lat1: Double, lon1: Double, lat2: Double, lon2: Double): Float {
        val phi1 = Math.toRadians(lat1)
        val phi2 = Math.toRadians(lat2)
        val deltaLambda = Math.toRadians(lon2 - lon1)

        val y = sin(deltaLambda) * cos(phi2)
        val x = cos(phi1) * sin(phi2) - sin(phi1) * cos(phi2) * cos(deltaLambda)
        val theta = atan2(y, x)
        val bearing = (Math.toDegrees(theta) + 360.0) % 360.0
        return bearing.toFloat()
    }

    /**
     * Calcula a diferen?a angular menor (0? a 180?) entre dois azimutes.
     */
    fun calculateAngularDifference(bearing1: Float, bearing2: Float): Float {
        val diff = Math.abs(bearing1 - bearing2) % 360f
        return if (diff > 180f) 360f - diff else diff
    }

    /**
     * Calcula o teto din?mico de toler?ncia de desvio baseado na velocidade atual e marge
m de tempo (SLA).
     */
    fun calculateDynamicMaxDeviationMeters(
        currentSpeedKmh: Double,
        slaMarginMinutes: Int
    ): Int {
        // Toler?ncia base urbana: 500 metros
        // Velocidades maiores (ex: 40-60 km/h em vias expressas) permitem maior desvio em
 metros
        val speedKmhClamped = currentSpeedKmh.coerceIn(8.0, 70.0)
        val speedBonus = (speedKmhClamped * 10.0).roundToInt()

        // Fator de SLA: margem curta de tempo reduz o limite toler?vel para proteger a no
ta
        val slaFactor = (slaMarginMinutes.toDouble() / 25.0).coerceIn(0.5, 1.5)
        val calculated = ((500 + speedBonus) * slaFactor).roundToInt()

        return calculated.coerceIn(350, 1400)
    }

    /**
     * Avalia o desvio geogr?fico completo de uma potencial mesclagem de pedidos a partir
do GPS atual do motorista.
     */
    fun evaluateMultiPointRouteDeviation(
        driverLat: Double,
        driverLng: Double,
        driverSpeedKmh: Double,
        o1: ScreenPoolOffer,
        o2: ScreenPoolOffer,
```

```
        slaMinutes: Int = 25
    ): HaversineDeviationResult {
        // Dist?ncia do motorista at? a primeira e segunda coleta
        val distToPickup1 = calculateHaversineDistanceMeters(driverLat, driverLng, o1.pick
upLat, o1.pickupLng).roundToInt()
        val distToPickup2 = calculateHaversineDistanceMeters(driverLat, driverLng, o2.pick
upLat, o2.pickupLng).roundToInt()

        // Desvio entre coletas e entregas (deslocamento intermedi?rio)
        val distBetweenPickups = calculateHaversineDistanceMeters(o1.pickupLat, o1.pickupL
ng, o2.pickupLat, o2.pickupLng).roundToInt()
        val distBetweenDropoffs = calculateHaversineDistanceMeters(o1.dropoffLat, o1.dropo
ffLng, o2.dropoffLat, o2.dropoffLng).roundToInt()

        // Rota direta do pedido mais longo (em metros)
        val directRoute1Meters = (o1.distanceKm * 1000.0).roundToInt()
        val directRoute2Meters = (o2.distanceKm * 1000.0).roundToInt()
        val maxDirectMeters = max(directRoute1Meters, directRoute2Meters)

        // Rota composta: soma da perna direta mais o desvio entre coletas e entregas (com
 fator de malha 1.18x)
        val extraTraveledMeters = ((distBetweenPickups + distBetweenDropoffs) * 1.18).roun
dToInt()
        val compositeMeters = maxDirectMeters + extraTraveledMeters

        // Desvio l?quido em metros
        val netDeviationMeters = extraTraveledMeters

        // Estimativa de tempo extra (calculado ? velocidade m?dia urbana de 26 km/h ou ve
locidade atual do GPS)
        val effectiveSpeedKmh = if (driverSpeedKmh > 12.0) driverSpeedKmh else 26.0
        val extraMinutes = (netDeviationMeters.toDouble() / (effectiveSpeedKmh * 1000.0 /
60.0))
            .let { if (it.isFinite()) it else 2.5 }
            .coerceAtLeast(1.0)

        // Limite din?mico ajustado por velocidade e SLA
        val dynamicMaxAllowed = calculateDynamicMaxDeviationMeters(driverSpeedKmh, slaMinu
tes)
        val isWithinTolerance = netDeviationMeters <= dynamicMaxAllowed

        // Azimute direcional dos dois pedidos
        val b1 = calculateBearing(o1.pickupLat, o1.pickupLng, o1.dropoffLat, o1.dropoffLng
)
        val b2 = calculateBearing(o2.pickupLat, o2.pickupLng, o2.dropoffLat, o2.dropoffLng
)
        val angularDiff = calculateAngularDifference(b1, b2)

        val status = when {
            netDeviationMeters <= (dynamicMaxAllowed * 0.55).roundToInt() && angularDiff <
= 35f -> DeviationStatusLevel.EXCELLENT
            isWithinTolerance && angularDiff <= 65f -> DeviationStatusLevel.ACCEPTABLE
            else -> DeviationStatusLevel.CRITICAL_EXCESSIVE
        }

        val explanation = when (status) {
            DeviationStatusLevel.EXCELLENT -> "Coletas a +${distBetweenPickups}m e entrega
s a +${distBetweenDropoffs}m no mesmo vetor (+${String.format(Locale.US, "%.1f", extraMinu
tes)} min)."
            DeviationStatusLevel.ACCEPTABLE -> "Desvio dentro da margem operacional de ${d
ynamicMaxAllowed}m (+${netDeviationMeters}m / +${String.format(Locale.US, "%.1f", extraMin
utes)} min)."
            DeviationStatusLevel.CRITICAL_EXCESSIVE -> "Desvio de ${netDeviationMeters}m e
xcede o limite din?mico seguro de ${dynamicMaxAllowed}m. Risco de atraso."
        }

        return HaversineDeviationResult(
            distanceDriverToPickup1Meters = distToPickup1,
            distanceDriverToPickup2Meters = distToPickup2,
            distancePickup1ToPickup2Meters = distBetweenPickups,
            distanceDropoff1ToDropoff2Meters = distBetweenDropoffs,
            directDistanceMeters = maxDirectMeters,
            compositeRouteDistanceMeters = compositeMeters,
            netDeviationMeters = netDeviationMeters,
            estimatedExtraMinutes = extraMinutes,
            dynamicMaxAllowedMeters = dynamicMaxAllowed,
            isWithinDynamicTolerance = isWithinTolerance,
            statusLevel = status,
            bearingDivergenceDeg = angularDiff,
            explanation = explanation
        )
    }
```

```
}

/**
 * Componente Composable visual para exibi??o do Desvio Geogr?fico Haversine.
 */
@Composable
fun HaversineDeviationBadge(
    result: HaversineDeviationResult,
    modifier: Modifier = Modifier
) {
    val statusColor = Color(result.statusLevel.colorHex)

    Surface(
        shape = RoundedCornerShape(10.dp),
        color = Color(0xFF0F172A),
        modifier = modifier
            .fillMaxWidth()
            .border(1.dp, statusColor.copy(alpha = 0.5f), RoundedCornerShape(10.dp))
    ) {
        Column(modifier = Modifier.padding(10.dp)) {
            Row(
                modifier = Modifier.fillMaxWidth(),
                verticalAlignment = Alignment.CenterVertically,
                horizontalArrangement = Arrangement.SpaceBetween
            ) {
                Row(verticalAlignment = Alignment.CenterVertically) {
                    Icon(
                        imageVector = when (result.statusLevel) {
                            DeviationStatusLevel.EXCELLENT -> Icons.Default.CheckCircle
                            DeviationStatusLevel.ACCEPTABLE -> Icons.Default.Route
                            DeviationStatusLevel.CRITICAL_EXCESSIVE -> Icons.Default.Warni
ng
                        },
                        contentDescription = null,
                        tint = statusColor,
                        modifier = Modifier.size(15.dp)
                    )
                    Spacer(modifier = Modifier.width(6.dp))
                    Text(
                        text = "DESVIO HAVERSINE: +${result.netDeviationMeters}m",
                        color = Color.White,
                        fontSize = 11.sp,
                        fontWeight = FontWeight.Black,
                        fontFamily = FontFamily.Monospace
                    )
                }

                Surface(
                    shape = RoundedCornerShape(4.dp),
                    color = statusColor.copy(alpha = 0.2f)
                ) {
                    Text(
                        text = "+${String.format(Locale.US, "%.1f", result.estimatedExtraM
inutes)} MIN",
                        color = statusColor,
                        fontSize = 10.sp,
                        fontWeight = FontWeight.Bold,
                        fontFamily = FontFamily.Monospace,
                        modifier = Modifier.padding(horizontal = 6.dp, vertical = 2.dp)
                    )
                }
            }

            Spacer(modifier = Modifier.height(6.dp))

            // Detalhamento de Coleta e Entrega
            Row(
                modifier = Modifier.fillMaxWidth(),
                horizontalArrangement = Arrangement.SpaceBetween
            ) {
                Text(
                    text = "Coletas: +${result.distancePickup1ToPickup2Meters}m",
                    color = Color(0xFF94A3B8),
                    fontSize = 10.sp
                )
                Text(
                    text = "Entregas: +${result.distanceDropoff1ToDropoff2Meters}m",
                    color = Color(0xFF94A3B8),
                    fontSize = 10.sp
                )
                Text(
                    text = "Teto SLA: ${result.dynamicMaxAllowedMeters}m",
```

```
                    color = Color(0xFF38BDF8),
                    fontSize = 10.sp,
                    fontWeight = FontWeight.Bold
                )
            }

            Spacer(modifier = Modifier.height(4.dp))

            Text(
                text = result.explanation,
                color = Color(0xFFCBD5E1),
                fontSize = 10.sp,
                lineHeight = 13.sp
            )
        }
    }
}
```

```
        private const val TAG = "SplitScreenCoordinator"

        @Volatile
        private var INSTANCE: SplitScreenDualRadarCoordinator? = null

        fun getInstance(context: Context): SplitScreenDualRadarCoordinator {
            return INSTANCE ?: synchronized(this) {
                INSTANCE ?: SplitScreenDualRadarCoordinator(context.applicationContext).al
so { INSTANCE = it }
            }
        }

        // Pares de aplicativos homologados para motoristas
        val PRECONFIGURED_PAIRS = listOf(
            SplitScreenAppPair(
                id = "ifood_99",
                title = "iFood (Topo) + 99Food (Base)",
                topPackage = "com.ifood.entregador",
                topAppName = "iFood Entregador",
                bottomPackage = "com.didiglobal.driver",
                bottomAppName = "99 Motorista / 99Food",
                description = "Par cl?ssico de alta densidade de pedidos em capitais e reg
i?es centrais."
            ),
            SplitScreenAppPair(
                id = "ifood_rappi",
                title = "iFood (Topo) + Rappi (Base)",
                topPackage = "com.ifood.entregador",
                topAppName = "iFood Entregador",
                bottomPackage = "com.grability.rappi.entregador",
                bottomAppName = "Rappi Entregador",
                description = "Ideal para zonas nobres e polos gastron?micos com alta conc
entra??o de hubs."
            ),
            SplitScreenAppPair(
                id = "noventa_rappi",
                title = "99Food (Topo) + Rappi (Base)",
                topPackage = "com.didiglobal.driver",
                topAppName = "99 Motorista",
                bottomPackage = "com.grability.rappi.entregador",
                bottomAppName = "Rappi Entregador",
                description = "Excelente alternativa para altern?ncia de hor?rios de pico
e dias de chuva."
            )
        )
    }

    private val _isSplitScreenActive = MutableStateFlow(false)
    val isSplitScreenActive: StateFlow<Boolean> = _isSplitScreenActive.asStateFlow()

    private val _topWindowTelemetry = MutableStateFlow<SplitWindowTelemetry?>(null)
    val topWindowTelemetry: StateFlow<SplitWindowTelemetry?> = _topWindowTelemetry.asState
Flow()

    private val _bottomWindowTelemetry = MutableStateFlow<SplitWindowTelemetry?>(null)
    val bottomWindowTelemetry: StateFlow<SplitWindowTelemetry?> = _bottomWindowTelemetry.a
sStateFlow()

    private val _selectedPair = MutableStateFlow(PRECONFIGURED_PAIRS[0])
    val selectedPair: StateFlow<SplitScreenAppPair> = _selectedPair.asStateFlow()

    fun selectPair(pair: SplitScreenAppPair) {
        _selectedPair.value = pair
    }

    /**
     * Varrer todas as janelas do sistema em tempo real capturadas pelo servi?o de acessib
ilidade.
     */
    fun processWindows(accessibilityService: AccessibilityService, windows: List<Accessibi
lityWindowInfo>?) {
        if (windows.isNullOrEmpty()) {
            _isSplitScreenActive.value = false
            return
        }

        // Filtra apenas janelas de aplicativos interativos
        val appWindows = windows.filter {
            it.type == AccessibilityWindowInfo.TYPE_APPLICATION && it.root != null
        }

        // Se houver pelo menos 2 janelas de aplicativos vis?veis simultaneamente, o modo
```

```
dividido est? ativo!
        val isDualActive = appWindows.size >= 2
        _isSplitScreenActive.value = isDualActive

        val screenBounds = Rect()
        val windowList = mutableListOf<SplitWindowTelemetry>()

        for (window in appWindows) {
            val root = window.root ?: continue
            val bounds = Rect()
            window.getBoundsInScreen(bounds)

            val pkg = root.packageName?.toString() ?: ""
            val appName = resolveAppName(pkg)

            // Classifica pela posi??o vertical (Y central)
            val centerY = bounds.centerY()
            val position = when {
                !isDualActive -> WindowVerticalPosition.FULL_SCREEN
                centerY < 1000 -> WindowVerticalPosition.TOP
                else -> WindowVerticalPosition.BOTTOM
            }

            // Extrai a oferta vis?vel nesta janela
            val offer = extractOfferFromWindowNode(pkg, appName, root)
            @Suppress("DEPRECATION")
            root.recycle()

            windowList.add(
                SplitWindowTelemetry(
                    position = position,
                    packageName = pkg,
                    appName = appName,
                    bounds = bounds,
                    latestOffer = offer,
                    isFocused = window.isFocused
                )
            )
        }

        val top = windowList.firstOrNull { it.position == WindowVerticalPosition.TOP }
        val bottom = windowList.firstOrNull { it.position == WindowVerticalPosition.BOTTOM
 }

        _topWindowTelemetry.value = top
        _bottomWindowTelemetry.value = bottom

        // Se ambas as metades possuem ofertas capturadas, alimenta o DeepOrderPoolMerging
Engine imediatamente
        if (top?.latestOffer != null && bottom?.latestOffer != null) {
            val poolEngine = DeepOrderPoolMergingEngine.getInstance(context)
            poolEngine.ingestPoolOffers(top.appName, listOf(top.latestOffer))
            poolEngine.ingestPoolOffers(bottom.appName, listOf(bottom.latestOffer))
            Log.i(TAG, "? Sucesso na leitura simult?nea: Topo=${top.appName} R$ ${top.late
stOffer.value} | Base=${bottom.appName} R$ ${bottom.latestOffer.value}")
        }
    }

    /**
     * Extrai a melhor oferta vis?vel dentro do n? de uma janela espec?fica
     */
    private fun extractOfferFromWindowNode(
        packageName: String,
        appName: String,
        rootNode: AccessibilityNodeInfo
    ): ScreenPoolOffer? {
        val texts = mutableListOf<String>()
        collectTexts(rootNode, texts)
        val blob = texts.joinToString(" ")

        val valueRegex = Regex("""R\$\s*(\d+[.,]\d{2})""")
        val distanceRegex = Regex("""(\d+[.,]?\d*)\s*km""", RegexOption.IGNORE_CASE)

        val matchVal = valueRegex.find(blob)
        val matchDist = distanceRegex.find(blob)

        if (matchVal != null && matchDist != null) {
            val v = matchVal.groupValues[1].replace(',', '.').toDoubleOrNull() ?: 0.0
            val d = matchDist.groupValues[1].replace(',', '.').toDoubleOrNull() ?: 0.0

            if (v > 0.0 && d > 0.0) {
                val rest = texts.firstOrNull {
```

```
                    it.contains("Restaurante", true) ||
                    it.contains("Shopping", true) ||
                    it.contains("Habib", true) ||
                    it.contains("Burger", true) ||
                    it.contains("Outback", true) ||
                    it.contains("Pizzaria", true)
                } ?: "Restaurante $appName"

                val drop = texts.firstOrNull {
                    it.contains("Rua", true) ||
                    it.contains("Av", true) ||
                    it.contains("Alameda", true)
                } ?: "Destino na Cidade"

                return ScreenPoolOffer(
                    appName = appName,
                    packageName = packageName,
                    restaurantName = rest,
                    pickupAddress = rest,
                    dropoffAddress = drop,
                    value = v,
                    distanceKm = d
                )
            }
        }
        return null
    }

    private fun collectTexts(node: AccessibilityNodeInfo?, list: MutableList<String>) {
        if (node == null) return
        val t = node.text?.toString()?.trim()
        val d = node.contentDescription?.toString()?.trim()
        if (!t.isNullOrEmpty()) list.add(t)
        else if (!d.isNullOrEmpty()) list.add(d)

        for (i in 0 until node.childCount) {
            val child = node.getChild(i)
            collectTexts(child, list)
            @Suppress("DEPRECATION")
            child?.recycle()
        }
    }

    /**
     * Dispara a a??o global do Android para abrir o modo de tela dividida.
     */
    fun triggerSplitScreenMode(accessibilityService: AccessibilityService?): Boolean {
        if (accessibilityService == null) {
            Log.w(TAG, "JarvisAccessibilityService n?o est? ativo.")
            return false
        }

        try {
            val success = accessibilityService.performGlobalAction(AccessibilityService.GL
OBAL_ACTION_TOGGLE_SPLIT_SCREEN)
            Log.i(TAG, "A??o GLOBAL_ACTION_TOGGLE_SPLIT_SCREEN disparada. Sucesso: $succes
s")
            return success
        } catch (e: Exception) {
            Log.e(TAG, "Falha ao acionar tela dividida via AccessibilityService", e)
            return false
        }
    }

    /**
     * Lan?a o par configurado de aplicativos.
     * Primeiro abre o app superior, aguarda brevemente e aciona a divis?o de tela para es
colher o inferior.
     */
    fun launchSplitScreenPair(context: Context, pair: SplitScreenAppPair) {
        try {
            val pm = context.packageManager
            val topIntent = pm.getLaunchIntentForPackage(pair.topPackage)
            if (topIntent != null) {
                topIntent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
                context.startActivity(topIntent)
            }

            // Solicita a divis?o via servi?o de acessibilidade se ativo
            JarvisAccessibilityService.instance?.let { service ->
                service.performGlobalAction(AccessibilityService.GLOBAL_ACTION_TOGGLE_SPLI
T_SCREEN)
```

```
            }

            TextToSpeechManager.getInstance(context).speakVoiceAlert(
                "Iniciando modo dividido: ${pair.topAppName} no topo e ${pair.bottomAppNam
e} na base. Leitura simult?nea ativada."
            )
        } catch (e: Exception) {
            Log.e(TAG, "Erro ao lan?ar par de tela dividida: ${e.message}")
        }
    }

    private fun resolveAppName(packageName: String): String {
        return when {
            packageName.contains("ifood", ignoreCase = true) -> "iFood"
            packageName.contains("didiglobal", ignoreCase = true) || packageName.contains(
"99", ignoreCase = true) -> "99Food"
            packageName.contains("rappi", ignoreCase = true) -> "Rappi"
            packageName.contains("uber", ignoreCase = true) -> "Uber Driver"
            packageName.contains("lalamove", ignoreCase = true) -> "Lalamove"
            else -> packageName.substringAfterLast('.')
        }
    }

    /**
     * Permite simular telemetria simult?nea em ambiente de teste/emulador.
     */
    fun simulateDualSplitWindowTelemetry() {
        val topOffer = ScreenPoolOffer(
            appName = "iFood",
            packageName = "com.ifood.entregador",
            restaurantName = "Fogo de Ch?o - Jardins",
            pickupAddress = "Rua Augusta, 2077",
            dropoffAddress = "Av. Rebou?as, 1800 - Pinheiros",
            value = 22.50,
            distanceKm = 3.4,
            pickupLat = -23.5590,
            pickupLng = -46.6620,
            dropoffLat = -23.5680,
            dropoffLng = -46.6850
        )

        val bottomOffer = ScreenPoolOffer(
            appName = "99Food",
            packageName = "com.didiglobal.driver",
            restaurantName = "Bullguer Jardins",
            pickupAddress = "Alameda Lorena, 1430",
            dropoffAddress = "Rua dos Pinheiros, 600 - Pinheiros",
            value = 18.00,
            distanceKm = 2.9,
            pickupLat = -23.5630,
            pickupLng = -46.6650,
            dropoffLat = -23.5670,
            dropoffLng = -46.6880
        )

        _isSplitScreenActive.value = true
        _topWindowTelemetry.value = SplitWindowTelemetry(
            position = WindowVerticalPosition.TOP,
            packageName = topOffer.packageName,
            appName = topOffer.appName,
            bounds = Rect(0, 0, 1080, 1150),
            latestOffer = topOffer,
            isFocused = true
        )

        _bottomWindowTelemetry.value = SplitWindowTelemetry(
            position = WindowVerticalPosition.BOTTOM,
            packageName = bottomOffer.packageName,
            appName = bottomOffer.appName,
            bounds = Rect(0, 1150, 1080, 2300),
            latestOffer = bottomOffer,
            isFocused = false
        )

        val poolEngine = DeepOrderPoolMergingEngine.getInstance(context)
        poolEngine.ingestPoolOffers("iFood", listOf(topOffer))
        poolEngine.ingestPoolOffers("99Food", listOf(bottomOffer))
    }
}
```

```
import android.animation.ValueAnimator
import android.annotation.SuppressLint
import android.content.Context
import android.content.Intent
import android.graphics.Color
import android.graphics.PixelFormat
import android.graphics.Typeface
import android.graphics.drawable.GradientDrawable
import android.net.Uri
import android.os.Build
import android.os.Handler
import android.os.Looper
import android.provider.Settings
import android.util.DisplayMetrics
import android.util.TypedValue
import android.view.Gravity
import android.view.MotionEvent
import android.view.View
import android.view.ViewConfiguration
import android.view.WindowManager
import android.widget.FrameLayout
import android.widget.ImageView
import android.widget.LinearLayout
import android.widget.TextView
import android.widget.Toast
import kotlin.math.hypot

/**
 * OverlayWindowManager
 *
 * Gerenciador nativo de interface flutuante (Bolha t?tica & Mini-HUD Compacto)
 * utilizando o [WindowManager] do Android.
 *
 * Funcionalidades da Bolha Flutuante T?tica Aprimorada (HUD sobre iFood/99/Uber):
 * 1. Bolha Flutuante Compacta persistente com indicador de rentabilidade em tempo real.
 * 2. Card Flutuante Compacto de Inspe??o de Corridas com c?lculo imediato de R$/km:
 *    - Ganho real por km (ex: R$ 7,20/km ? BOA ou R$ 2,80/km ? PREJU?ZO).
 *    - Indicador visual luminoso de Rota Dupla Compat?vel (sobreposi??o de pedidos).
 *    - A??es em 1 Toque: Bot?es gigantes t?teis de ACEITAR ou DISPENSAR sem precisar alte
rnar manualmente.
 * 3. Ancoragem magn?tica suave nas bordas laterais da tela (Snap-to-Edge).
 */
class OverlayWindowManager private constructor(private val context: Context) {

    data class JarvisOverlayStatus(
        val isOnline: Boolean = true,
        val headline: String = "JARVIS OPERACIONAL",
        val activeApp: String = "Monitorando Apps",
        val profitPerKm: String = "R$ 4,50/km",
        val blitzAlert: String = "Raio Seguro (Sem Blitz)",
        val latestOffer: String? = null,
        val alertLevel: AlertLevel = AlertLevel.NORMAL
    ) {
        enum class AlertLevel {
            NORMAL,      // Verde Neon
            OPPORTUNITY, // Azul / Ciano
            WARNING      // Laranja / Vermelho
        }
    }

    /**
     * Modelo da Corrida Detectada na Bolha Flutuante
     */
    data class FloatingTacticalOffer(
        val appName: String = "iFood",
        val packageName: String = "com.ifood.driver",
        val value: Double = 28.50,
        val distanceKm: Double = 3.9,
        val pricePerKm: Double = 7.30,
        val isGoodDeal: Boolean = true,
        val dealLabel: String = "R$ 7,20/km ? BOA",
        val destination: String = "Av. Rebou?as, 1200",
        val pickup: String = "Shopping Eldorado",
        val hasDualRoute: Boolean = true,
        val dualRouteAppName: String = "99 Moto",
        val dualRouteExtraGain: Double = 16.50,
        val haversineDeviationMeters: Int = 0,
        val haversineExtraMinutes: Double = 0.0,
        val haversineExplanation: String = ""
    )

    private val windowManager: WindowManager =
```

```
        context.getSystemService(Context.WINDOW_SERVICE) as WindowManager

    private val mainHandler = Handler(Looper.getMainLooper())
    private val touchSlop = ViewConfiguration.get(context).scaledTouchSlop

    private var overlayRootView: FrameLayout? = null
    private var bubbleContainer: FrameLayout? = null
    private var hudContainer: LinearLayout? = null

    // Componentes da Bolha Circular
    private var tvBubbleBadge: TextView? = null
    private var ivBubblePulse: View? = null

    // Componentes do Card Flutuante de Corrida
    private var offerCardBox: LinearLayout? = null
    private var tvOfferAppTag: TextView? = null
    private var tvOfferProfitMain: TextView? = null
    private var tvOfferProfitSub: TextView? = null
    private var tvOfferMetrics: TextView? = null
    private var tvOfferAddresses: TextView? = null
    private var dualRouteBox: LinearLayout? = null
    private var tvDualRouteText: TextView? = null

    // Componentes de Desvio Geod?sico Haversine no HUD
    private var haversineBox: LinearLayout? = null
    private var tvHaversineTitle: TextView? = null
    private var tvHaversineBadge: TextView? = null
    private var tvHaversineDetails: TextView? = null

    private var btnAcceptRide: TextView? = null
    private var btnDeclineRide: TextView? = null
    private var btnSwitchApp: TextView? = null

    // Componentes de Status Geral
    private var tvHudTitle: TextView? = null
    private var tvHudSubstatus: TextView? = null
    private var tvHudProfit: TextView? = null
    private var tvHudBlitz: TextView? = null
    private var defaultMetricsBox: LinearLayout? = null

    private var layoutParams: WindowManager.LayoutParams? = null

    private var isExpanded = false
    private var isShowing = false
    private var currentStatus = JarvisOverlayStatus()
    private var currentOffer: FloatingTacticalOffer? = null

    // Vari?veis para c?lculo de arraste
    private var initialX = 0
    private var initialY = 0
    private var initialTouchX = 0f
    private var initialTouchY = 0f
    private var isDragging = false
    private var touchStartTime: Long = 0

    companion object {
        @SuppressLint("StaticFieldLeak")
        @Volatile
        private var INSTANCE: OverlayWindowManager? = null

        fun getInstance(context: Context): OverlayWindowManager {
            return INSTANCE ?: synchronized(this) {
                INSTANCE ?: OverlayWindowManager(context.applicationContext).also { INSTAN
CE = it }
            }
        }

        fun canDrawOverlays(context: Context): Boolean {
            return Settings.canDrawOverlays(context)
        }

        fun requestOverlayPermission(context: Context) {
            val intent = Intent(
                Settings.ACTION_MANAGE_OVERLAY_PERMISSION,
                Uri.parse("package:${context.packageName}")
            ).apply {
                addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
            }
            context.startActivity(intent)
        }
    }
```

```
    /**
     * Exibe ou atualiza a interface flutuante sobreposta aos outros apps.
     */
    fun show() {
        if (!canDrawOverlays(context)) return

        mainHandler.post {
            if (isShowing && overlayRootView != null) {
                overlayRootView?.visibility = View.VISIBLE
                return@post
            }
            createAndAttachOverlayView()
        }
    }

    /**
     * Oculta a interface flutuante.
     */
    fun hide() {
        mainHandler.post {
            overlayRootView?.visibility = View.GONE
        }
    }

    fun toggle() {
        if (isShowing && overlayRootView?.visibility == View.VISIBLE) {
            hide()
        } else {
            show()
        }
    }

    fun isShowing(): Boolean = isShowing && overlayRootView?.visibility == View.VISIBLE

    /**
     * Atualiza o estado geral exibido na bolha.
     */
    fun updateStatus(status: JarvisOverlayStatus) {
        this.currentStatus = status
        mainHandler.post {
            applyStatusToViews()
        }
    }

    /**
     * Dispara o Card Flutuante Compacto de Inspe??o de Corrida com c?lculo de R$/km e Rot
a Dupla
     */
    fun showTacticalOffer(offer: FloatingTacticalOffer) {
        this.currentOffer = offer
        mainHandler.post {
            if (!isShowing || overlayRootView == null) {
                show()
            }
            // For?a a expans?o do HUD para visualiza??o imediata pelo motorista
            setExpandedState(true)
            applyOfferToViews(offer)

            // Resposta t?til e sonora
            try {
                if (offer.isGoodDeal) {
                    HapticFeedbackHelper.vibrateSuccess(context)
                } else {
                    HapticFeedbackHelper.vibrateWarning(context)
                }
            } catch (e: Exception) {
                e.printStackTrace()
            }

            // Leitura autom?tica em voz alta do resumo da oferta (valor, dist?ncia e lucr
o/km)
            try {
                OfferTextToSpeechEngine.getInstance(context).speakTacticalOffer(offer)
            } catch (e: Exception) {
                e.printStackTrace()
            }
        }
    }

    /**
     * Exibe alerta flutuante de cancelamento parcial em corrida mesclada sobre os apps de
 entrega
```

```
     */
    fun showCancellationAlert(event: StackCancellationEvent) {
        val tacticalOffer = FloatingTacticalOffer(
            appName = event.appName,
            packageName = "com.ifood.driver",
            value = event.remainingValue,
            distanceKm = event.remainingDistanceKm,
            pricePerKm = event.remainingGainPerKm,
            isGoodDeal = event.isViable,
            dealLabel = if (event.isViable) "?? SUB-PEDIDO CANCELADO ? RESTANTE VI?VEL" el
se "? ALERTA: RESTANTE PREJUDICIAL",
            destination = event.remainingRestaurants,
            pickup = "Cancelado: ${event.cancelledRestaurant} (-R$ ${String.format(java.ut
il.Locale.ROOT, "%.2f", event.cancelledValue)})",
            hasDualRoute = false
        )
        showTacticalOffer(tacticalOffer)
    }

    /**
     * Limpa a oferta atual e recolhe o HUD
     */
    fun clearTacticalOffer() {
        this.currentOffer = null
        mainHandler.post {
            offerCardBox?.visibility = View.GONE
            defaultMetricsBox?.visibility = View.VISIBLE
            setExpandedState(false)
            applyStatusToViews()
        }
    }

    /**
     * Remove a sobreposi??o do WindowManager.
     */
    fun destroy() {
        mainHandler.post {
            overlayRootView?.let { view ->
                if (view.isAttachedToWindow) {
                    try {
                        windowManager.removeViewImmediate(view)
                    } catch (e: Exception) {
                        e.printStackTrace()
                    }
                }
            }
            overlayRootView = null
            bubbleContainer = null
            hudContainer = null
            isShowing = false
            isExpanded = false
            currentOffer = null
        }
    }

    // ================================================================================
    // CONSTRU??O E ANCORAGEM DA VIEW NATIVA
    // ================================================================================

    @SuppressLint("ClickableViewAccessibility")
    private fun createAndAttachOverlayView() {
        val windowType = if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            WindowManager.LayoutParams.TYPE_APPLICATION_OVERLAY
        } else {
            @Suppress("DEPRECATION")
            WindowManager.LayoutParams.TYPE_PHONE
        }

        val displayMetrics = context.resources.displayMetrics
        val screenWidth = displayMetrics.widthPixels
        val startX = screenWidth - dpToPx(76)
        val startY = dpToPx(160)

        val params = WindowManager.LayoutParams(
            WindowManager.LayoutParams.WRAP_CONTENT,
            WindowManager.LayoutParams.WRAP_CONTENT,
            windowType,
            WindowManager.LayoutParams.FLAG_NOT_FOCUSABLE or
                    WindowManager.LayoutParams.FLAG_LAYOUT_NO_LIMITS,
            PixelFormat.TRANSLUCENT
        ).apply {
            gravity = Gravity.TOP or Gravity.START
```

```
            x = startX
            y = startY
        }
        this.layoutParams = params

        val root = FrameLayout(context).apply {
            clipChildren = false
            clipToPadding = false
        }

        val bubble = buildBubbleView()
        this.bubbleContainer = bubble

        val hud = buildHudView()
        this.hudContainer = hud

        root.addView(bubble)
        root.addView(hud)

        bubble.setOnTouchListener { _, event ->
            handleBubbleTouch(event)
        }

        this.overlayRootView = root

        try {
            windowManager.addView(root, params)
            isShowing = true
            isExpanded = false
            setExpandedState(false)
            applyStatusToViews()
        } catch (e: Exception) {
            e.printStackTrace()
            isShowing = false
        }
    }

    /**
     * Monta a bolha circular compacta
     */
    private fun buildBubbleView(): FrameLayout {
        val bubbleSize = dpToPx(62)

        val container = FrameLayout(context).apply {
            layoutParams = FrameLayout.LayoutParams(bubbleSize, bubbleSize)
            background = GradientDrawable().apply {
                shape = GradientDrawable.OVAL
                setColor(Color.parseColor("#10151E"))
                setStroke(dpToPx(2.2f), Color.parseColor("#00E676"))
            }
            elevation = dpToPx(10).toFloat()
        }

        val pulse = View(context).apply {
            val pulseSize = dpToPx(11)
            layoutParams = FrameLayout.LayoutParams(pulseSize, pulseSize).apply {
                gravity = Gravity.TOP or Gravity.END
                topMargin = dpToPx(5)
                rightMargin = dpToPx(5)
            }
            background = GradientDrawable().apply {
                shape = GradientDrawable.OVAL
                setColor(Color.parseColor("#00E676"))
            }
        }
        this.ivBubblePulse = pulse
        container.addView(pulse)

        val tvIcon = TextView(context).apply {
            layoutParams = FrameLayout.LayoutParams(
                FrameLayout.LayoutParams.MATCH_PARENT,
                FrameLayout.LayoutParams.MATCH_PARENT
            )
            gravity = Gravity.CENTER
            text = "?"
            textSize = 24f
        }
        container.addView(tvIcon)

        val tvBadge = TextView(context).apply {
            val badgeHeight = dpToPx(18)
            layoutParams = FrameLayout.LayoutParams(
```

```
                FrameLayout.LayoutParams.WRAP_CONTENT,
                badgeHeight
            ).apply {
                gravity = Gravity.BOTTOM or Gravity.CENTER_HORIZONTAL
                bottomMargin = dpToPx(2)
            }
            background = GradientDrawable().apply {
                shape = GradientDrawable.RECTANGLE
                cornerRadius = dpToPx(9).toFloat()
                setColor(Color.parseColor("#00E676"))
            }
            setPadding(dpToPx(6), 0, dpToPx(6), 0)
            text = "JARVIS"
            setTextColor(Color.parseColor("#0A0E14"))
            textSize = 8.5f
            typeface = Typeface.DEFAULT_BOLD
            gravity = Gravity.CENTER
        }
        this.tvBubbleBadge = tvBadge
        container.addView(tvBadge)

        return container
    }

    /**
     * Monta o HUD t?tico compacto completo com a se??o de inspe??o de corridas
     */
    private fun buildHudView(): LinearLayout {
        val hudWidth = dpToPx(310)

        val container = LinearLayout(context).apply {
            layoutParams = FrameLayout.LayoutParams(
                hudWidth,
                FrameLayout.LayoutParams.WRAP_CONTENT
            )
            orientation = LinearLayout.VERTICAL
            visibility = View.GONE
            setPadding(dpToPx(16), dpToPx(14), dpToPx(16), dpToPx(14))
            background = GradientDrawable().apply {
                shape = GradientDrawable.RECTANGLE
                cornerRadius = dpToPx(20).toFloat()
                setColor(Color.parseColor("#F40A0D14"))
                setStroke(dpToPx(1.5f), Color.parseColor("#00E676"))
            }
            elevation = dpToPx(14).toFloat()
        }

        // Cabe?alho: T?tulo + Bot?o Fechar
        val headerRow = LinearLayout(context).apply {
            layoutParams = LinearLayout.LayoutParams(
                LinearLayout.LayoutParams.MATCH_PARENT,
                LinearLayout.LayoutParams.WRAP_CONTENT
            )
            orientation = LinearLayout.HORIZONTAL
            gravity = Gravity.CENTER_VERTICAL
        }

        val tvTitle = TextView(context).apply {
            layoutParams = LinearLayout.LayoutParams(0, LinearLayout.LayoutParams.WRAP_CON
TENT, 1f)
            text = "HUD T?TICO COCKPIT"
            setTextColor(Color.WHITE)
            textSize = 13.spToSp()
            typeface = Typeface.DEFAULT_BOLD
        }
        this.tvHudTitle = tvTitle
        headerRow.addView(tvTitle)

        // Bot?o de ?udio TTS (Ouvir resumo em voz alta / Mudo)
        val btnSpeaker = TextView(context).apply {
            layoutParams = LinearLayout.LayoutParams(dpToPx(30), dpToPx(30)).apply {
                rightMargin = dpToPx(6)
            }
            gravity = Gravity.CENTER
            text = "?"
            textSize = 14f
            background = GradientDrawable().apply {
                shape = GradientDrawable.OVAL
                setColor(Color.parseColor("#1C2533"))
            }
            setOnClickListener {
                val ttsEngine = OfferTextToSpeechEngine.getInstance(context)
```

```
                val offer = currentOffer
                if (offer != null) {
                    ttsEngine.speakTacticalOffer(offer)
                    Toast.makeText(context, "? Lendo resumo da oferta em voz alta...", Toa
st.LENGTH_SHORT).show()
                } else {
                    val repeated = ttsEngine.repeatLastOffer()
                    if (!repeated) {
                        ttsEngine.speak("Nenhuma oferta recente para leitura.")
                    }
                    Toast.makeText(context, "? Reproduzindo ?udio TTS...", Toast.LENGTH_SH
ORT).show()
                }
            }
            setOnLongClickListener {
                val ttsEngine = OfferTextToSpeechEngine.getInstance(context)
                val isMuted = ttsEngine.toggleMute()
                text = if (isMuted) "?" else "?"
                Toast.makeText(
                    context,
                    if (isMuted) "Voz TTS silenciada" else "Voz TTS ativada",
                    Toast.LENGTH_SHORT
                ).show()
                true
            }
        }
        headerRow.addView(btnSpeaker)

        val btnClose = TextView(context).apply {
            layoutParams = LinearLayout.LayoutParams(dpToPx(30), dpToPx(30))
            gravity = Gravity.CENTER
            text = "?"
            setTextColor(Color.parseColor("#90A4AE"))
            textSize = 15f
            typeface = Typeface.DEFAULT_BOLD
            setOnClickListener {
                setExpandedState(false)
            }
        }
        headerRow.addView(btnClose)
        container.addView(headerRow)

        // Substatus / Aplicativos monitorados
        val tvSub = TextView(context).apply {
            layoutParams = LinearLayout.LayoutParams(
                LinearLayout.LayoutParams.MATCH_PARENT,
                LinearLayout.LayoutParams.WRAP_CONTENT
            ).apply {
                topMargin = dpToPx(2)
                bottomMargin = dpToPx(8)
            }
            text = "? Monitorando: iFood ? Uber ? 99 Moto"
            setTextColor(Color.parseColor("#00E676"))
            textSize = 10.5f
        }
        this.tvHudSubstatus = tvSub
        container.addView(tvSub)

        // ========================================================================
        // CARD FLUTUANTE COMPACTO DE OFERTA (CORRIDA DETECTADA)
        // ========================================================================
        val offerBox = LinearLayout(context).apply {
            layoutParams = LinearLayout.LayoutParams(
                LinearLayout.LayoutParams.MATCH_PARENT,
                LinearLayout.LayoutParams.WRAP_CONTENT
            ).apply {
                bottomMargin = dpToPx(10)
            }
            orientation = LinearLayout.VERTICAL
            visibility = View.GONE
            setPadding(dpToPx(12), dpToPx(10), dpToPx(12), dpToPx(10))
            background = GradientDrawable().apply {
                shape = GradientDrawable.RECTANGLE
                cornerRadius = dpToPx(14).toFloat()
                setColor(Color.parseColor("#151B26"))
                setStroke(dpToPx(1.5f), Color.parseColor("#00E676"))
            }
        }
        this.offerCardBox = offerBox

        // Linha 1 do Card: Tag da Plataforma (ex: iFood ?) + Timer
        val tagRow = LinearLayout(context).apply {
```

```
        layoutParams = LinearLayout.LayoutParams(
            LinearLayout.LayoutParams.MATCH_PARENT,
            LinearLayout.LayoutParams.WRAP_CONTENT
        )
        orientation = LinearLayout.HORIZONTAL
        gravity = Gravity.CENTER_VERTICAL
    }

    val tvAppTag = TextView(context).apply {
        layoutParams = LinearLayout.LayoutParams(
            LinearLayout.LayoutParams.WRAP_CONTENT,
            LinearLayout.LayoutParams.WRAP_CONTENT
        )
        background = GradientDrawable().apply {
            shape = GradientDrawable.RECTANGLE
            cornerRadius = dpToPx(6).toFloat()
            setColor(Color.parseColor("#EA1D2C"))
        }
        setPadding(dpToPx(8), dpToPx(3), dpToPx(8), dpToPx(3))
        text = "iFood"
        setTextColor(Color.WHITE)
        textSize = 10f
        typeface = Typeface.DEFAULT_BOLD
    }
    this.tvOfferAppTag = tvAppTag
    tagRow.addView(tvAppTag)

    val spacerTag = View(context).apply {
        layoutParams = LinearLayout.LayoutParams(0, 1, 1f)
    }
    tagRow.addView(spacerTag)

    val tvTimer = TextView(context).apply {
        text = "? 15s p/ aceitar"
        setTextColor(Color.parseColor("#FFD700"))
        textSize = 10f
        typeface = Typeface.DEFAULT_BOLD
    }
    tagRow.addView(tvTimer)
    offerBox.addView(tagRow)

    // Linha 2 do Card: GANHO REAL POR KM (Destaque Principal)
    val profitBanner = LinearLayout(context).apply {
        layoutParams = LinearLayout.LayoutParams(
            LinearLayout.LayoutParams.MATCH_PARENT,
            LinearLayout.LayoutParams.WRAP_CONTENT
        ).apply {
            topMargin = dpToPx(8)
            bottomMargin = dpToPx(6)
        }
        orientation = LinearLayout.VERTICAL
        setPadding(dpToPx(10), dpToPx(8), dpToPx(10), dpToPx(8))
        background = GradientDrawable().apply {
            shape = GradientDrawable.RECTANGLE
            cornerRadius = dpToPx(10).toFloat()
            setColor(Color.parseColor("#122A1E"))
            setStroke(dpToPx(1.2f), Color.parseColor("#00E676"))
        }
    }

    val tvProfitMain = TextView(context).apply {
        text = "R$ 7,20/km ? BOA"
        setTextColor(Color.parseColor("#00FF88"))
        textSize = 17f
        typeface = Typeface.DEFAULT_BOLD
        gravity = Gravity.CENTER
    }
    this.tvOfferProfitMain = tvProfitMain
    profitBanner.addView(tvProfitMain)

    val tvProfitSub = TextView(context).apply {
        text = "Excelente rentabilidade ? Acima do piso"
        setTextColor(Color.parseColor("#B0BEC5"))
        textSize = 10f
        gravity = Gravity.CENTER
    }
    this.tvOfferProfitSub = tvProfitSub
    profitBanner.addView(tvProfitSub)
    offerBox.addView(profitBanner)

    // Linha 3 do Card: M?tricas (Valor Total, Dist?ncia)
    val tvMetrics = TextView(context).apply {
```

```
            layoutParams = LinearLayout.LayoutParams(
                LinearLayout.LayoutParams.MATCH_PARENT,
                LinearLayout.LayoutParams.WRAP_CONTENT
            ).apply {
                bottomMargin = dpToPx(4)
            }
            text = "? R$ 28,50 ? ? 3,9 km ? ? 12 min"
            setTextColor(Color.WHITE)
            textSize = 12f
            typeface = Typeface.DEFAULT_BOLD
        }
        this.tvOfferMetrics = tvMetrics
        offerBox.addView(tvMetrics)

        // Linha 4 do Card: Destino e Coleta
        val tvAddresses = TextView(context).apply {
            layoutParams = LinearLayout.LayoutParams(
                LinearLayout.LayoutParams.MATCH_PARENT,
                LinearLayout.LayoutParams.WRAP_CONTENT
            ).apply {
                bottomMargin = dpToPx(8)
            }
            text = "? De: Shopping Eldorado ? Para: Av. Rebou?as"
            setTextColor(Color.parseColor("#90A4AE"))
            textSize = 10.5f
            maxLines = 2
        }
        this.tvOfferAddresses = tvAddresses
        offerBox.addView(tvAddresses)

        // Linha 5: INDICADOR DE ROTA DUPLA COMPAT?VEL
        val dualBox = LinearLayout(context).apply {
            layoutParams = LinearLayout.LayoutParams(
                LinearLayout.LayoutParams.MATCH_PARENT,
                LinearLayout.LayoutParams.WRAP_CONTENT
            ).apply {
                bottomMargin = dpToPx(10)
            }
            orientation = LinearLayout.HORIZONTAL
            setPadding(dpToPx(8), dpToPx(6), dpToPx(8), dpToPx(6))
            background = GradientDrawable().apply {
                shape = GradientDrawable.RECTANGLE
                cornerRadius = dpToPx(8).toFloat()
                setColor(Color.parseColor("#0C2030"))
                setStroke(dpToPx(1f), Color.parseColor("#00E5FF"))
            }
            gravity = Gravity.CENTER_VERTICAL
        }
        this.dualRouteBox = dualBox

        val tvDual = TextView(context).apply {
            text = "? ROTA DUPLA COMPAT?VEL! (+R$ 16,50 c/ 99 Moto)"
            setTextColor(Color.parseColor("#00E5FF"))
            textSize = 10.5f
            typeface = Typeface.DEFAULT_BOLD
        }
        this.tvDualRouteText = tvDual
        dualBox.addView(tvDual)
        offerBox.addView(dualBox)

        // Linha 5.1: INDICADOR VISUAL DE DESVIO HAVERSINE (METROS + MINUTOS EXTRAS NO HUD
)
        val hBox = LinearLayout(context).apply {
            layoutParams = LinearLayout.LayoutParams(
                LinearLayout.LayoutParams.MATCH_PARENT,
                LinearLayout.LayoutParams.WRAP_CONTENT
            ).apply {
                bottomMargin = dpToPx(8)
            }
            orientation = LinearLayout.VERTICAL
            setPadding(dpToPx(8), dpToPx(6), dpToPx(8), dpToPx(6))
            background = GradientDrawable().apply {
                shape = GradientDrawable.RECTANGLE
                cornerRadius = dpToPx(8).toFloat()
                setColor(Color.parseColor("#0B132B"))
                setStroke(dpToPx(1f), Color.parseColor("#10B981"))
            }
        }
        this.haversineBox = hBox

        val hTitle = TextView(context).apply {
            text = "? DESVIO HAVERSINE: +380m ? +1.8 min extras"
```

```
    setTextColor(Color.parseColor("#38BDF8"))
    textSize = 10.5f
    typeface = Typeface.DEFAULT_BOLD
}
this.tvHaversineTitle = hTitle
hBox.addView(hTitle)

val hBadge = TextView(context).apply {
    text = "[? DESVIO M?NIMO / SEGURO]"
    setTextColor(Color.parseColor("#10B981"))
    textSize = 10f
    typeface = Typeface.DEFAULT_BOLD
}
this.tvHaversineBadge = hBadge
hBox.addView(hBadge)

val hDetails = TextView(context).apply {
    text = "Toler?ncia din?mica: 820m (Velocidade e SLA seguros)"
    setTextColor(Color.parseColor("#94A3B8"))
    textSize = 9.5f
}
this.tvHaversineDetails = hDetails
hBox.addView(hDetails)

offerBox.addView(hBox)

// Banner Viva-Voz no HUD Flutuante (Speech-to-Text Ativo)
val voiceRow = LinearLayout(context).apply {
    layoutParams = LinearLayout.LayoutParams(
        LinearLayout.LayoutParams.MATCH_PARENT,
        LinearLayout.LayoutParams.WRAP_CONTENT
    ).apply {
        bottomMargin = dpToPx(8)
    }
    orientation = LinearLayout.HORIZONTAL
    setPadding(dpToPx(8), dpToPx(5), dpToPx(8), dpToPx(5))
    background = GradientDrawable().apply {
        shape = GradientDrawable.RECTANGLE
        cornerRadius = dpToPx(8).toFloat()
        setColor(Color.parseColor("#0F1E19"))
        setStroke(dpToPx(1f), Color.parseColor("#00FF88"))
    }
    gravity = Gravity.CENTER_VERTICAL
}

val tvVoiceStatus = TextView(context).apply {
    text = "? VIVA-VOZ: Diga 'ACEITAR' ou 'RECUSAR'"
    setTextColor(Color.parseColor("#00FF88"))
    textSize = 10f
    typeface = Typeface.DEFAULT_BOLD
}
voiceRow.addView(tvVoiceStatus)
offerBox.addView(voiceRow)

// ========================================================================
// BOT?ES GIGANTES DE A??O EM 1 TOQUE (ACEITAR OU DISPENSAR)
// ========================================================================
val oneTouchActionsRow = LinearLayout(context).apply {
    layoutParams = LinearLayout.LayoutParams(
        LinearLayout.LayoutParams.MATCH_PARENT,
        LinearLayout.LayoutParams.WRAP_CONTENT
    )
    orientation = LinearLayout.HORIZONTAL
}

// Bot?o Gigante 1: ACEITAR CORRIDA ?
val btnAccept = TextView(context).apply {
    layoutParams = LinearLayout.LayoutParams(0, dpToPx(48), 1.2f).apply {
        rightMargin = dpToPx(6)
    }
    gravity = Gravity.CENTER
    background = GradientDrawable().apply {
        shape = GradientDrawable.RECTANGLE
        cornerRadius = dpToPx(12).toFloat()
        setColor(Color.parseColor("#00E676"))
    }
    text = "ACEITAR ?"
    setTextColor(Color.parseColor("#0A0E14"))
    textSize = 13f
    typeface = Typeface.DEFAULT_BOLD
    setOnClickListener {
        handleAcceptCurrentOffer()
```

```
        }
    }
    this.btnAcceptRide = btnAccept
    oneTouchActionsRow.addView(btnAccept)

    // Bot?o T?tico: ROTA ?? (Waze / Google Maps em 1 toque)
    val btnRoute = TextView(context).apply {
        layoutParams = LinearLayout.LayoutParams(0, dpToPx(48), 0.9f).apply {
            leftMargin = dpToPx(4)
            rightMargin = dpToPx(4)
        }
        gravity = Gravity.CENTER
        background = GradientDrawable().apply {
            shape = GradientDrawable.RECTANGLE
            cornerRadius = dpToPx(12).toFloat()
            setColor(Color.parseColor("#0C2030"))
            setStroke(dpToPx(1.2f), Color.parseColor("#00E5FF"))
        }
        text = "ROTA ??"
        setTextColor(Color.parseColor("#00E5FF"))
        textSize = 12f
        typeface = Typeface.DEFAULT_BOLD
        setOnClickListener {
            val offer = currentOffer
            val targetAddress = offer?.pickup ?: "Ponto de Coleta"
            NavigationQuickDispatcher.launchBestRoute(
                context = context,
                latitude = -23.5616,
                longitude = -46.6559,
                addressTitle = targetAddress,
                preferWaze = true
            )
        }
    }
    oneTouchActionsRow.addView(btnRoute)

    // Bot?o Gigante 2: DISPENSAR ?
    val btnDecline = TextView(context).apply {
        layoutParams = LinearLayout.LayoutParams(0, dpToPx(48), 1f).apply {
            leftMargin = dpToPx(4)
        }
        gravity = Gravity.CENTER
        background = GradientDrawable().apply {
            shape = GradientDrawable.RECTANGLE
            cornerRadius = dpToPx(12).toFloat()
            setColor(Color.parseColor("#2A1218"))
            setStroke(dpToPx(1.2f), Color.parseColor("#FF3366"))
        }
        text = "DISPENSAR ?"
        setTextColor(Color.parseColor("#FF5252"))
        textSize = 12f
        typeface = Typeface.DEFAULT_BOLD
        setOnClickListener {
            handleDeclineCurrentOffer()
        }
    }
    this.btnDeclineRide = btnDecline
    oneTouchActionsRow.addView(btnDecline)

    offerBox.addView(oneTouchActionsRow)

    // Bot?o T?tico de Tela Dividida em 1 Toque na Oferta (Guid?o da Moto)
    val btnSplitInOffer = TextView(context).apply {
        layoutParams = LinearLayout.LayoutParams(
            LinearLayout.LayoutParams.MATCH_PARENT,
            dpToPx(40)
        ).apply {
            topMargin = dpToPx(8)
        }
        gravity = Gravity.CENTER
        background = GradientDrawable().apply {
            shape = GradientDrawable.RECTANGLE
            cornerRadius = dpToPx(10).toFloat()
            setColor(Color.parseColor("#1E1B4B"))
            setStroke(dpToPx(1.2f), Color.parseColor("#6366F1"))
        }
        text = "? TELA DIVIDIDA (1 TOQUE - iFood ?? / 99 ??)"
        setTextColor(Color.parseColor("#C7D2FE"))
        textSize = 11.5f
        typeface = Typeface.DEFAULT_BOLD
        setOnClickListener {
            val coordinator = SplitScreenDualRadarCoordinator.getInstance(context)
```

```
                val pair = coordinator.selectedPair.value
                coordinator.launchSplitScreenPair(context, pair)
                Toast.makeText(context, "? Lan?ando Tela Dividida: ${pair.topAppName} (Top
o) + ${pair.bottomAppName} (Base)", Toast.LENGTH_LONG).show()
            }
        }
        offerBox.addView(btnSplitInOffer)
        container.addView(offerBox)

        // ========================================================================
        // M?TRICAS PADR?O QUANDO N?O H? CORRIDA ATIVA
        // ========================================================================
        val defaultBox = LinearLayout(context).apply {
            layoutParams = LinearLayout.LayoutParams(
                LinearLayout.LayoutParams.MATCH_PARENT,
                LinearLayout.LayoutParams.WRAP_CONTENT
            )
            orientation = LinearLayout.VERTICAL
        }
        this.defaultMetricsBox = defaultBox

        val metricsRow = LinearLayout(context).apply {
            layoutParams = LinearLayout.LayoutParams(
                LinearLayout.LayoutParams.MATCH_PARENT,
                LinearLayout.LayoutParams.WRAP_CONTENT
            )
            orientation = LinearLayout.HORIZONTAL
        }

        val profitBox = LinearLayout(context).apply {
            layoutParams = LinearLayout.LayoutParams(0, LinearLayout.LayoutParams.WRAP_CON
TENT, 1f)
            orientation = LinearLayout.VERTICAL
        }
        val lblProfit = TextView(context).apply {
            text = "PISO M?NIMO"
            setTextColor(Color.parseColor("#90A4AE"))
            textSize = 9f
        }
        val valProfit = TextView(context).apply {
            text = "R$ 4,50/km"
            setTextColor(Color.parseColor("#00E676"))
            textSize = 13f
            typeface = Typeface.DEFAULT_BOLD
        }
        this.tvHudProfit = valProfit
        profitBox.addView(lblProfit)
        profitBox.addView(valProfit)
        metricsRow.addView(profitBox)

        val blitzBox = LinearLayout(context).apply {
            layoutParams = LinearLayout.LayoutParams(0, LinearLayout.LayoutParams.WRAP_CON
TENT, 1.2f)
            orientation = LinearLayout.VERTICAL
        }
        val lblBlitz = TextView(context).apply {
            text = "RADAR / BLITZ"
            setTextColor(Color.parseColor("#90A4AE"))
            textSize = 9f
        }
        val valBlitz = TextView(context).apply {
            text = "Raio Seguro"
            setTextColor(Color.parseColor("#81C784"))
            textSize = 12f
            typeface = Typeface.DEFAULT_BOLD
        }
        this.tvHudBlitz = valBlitz
        blitzBox.addView(lblBlitz)
        blitzBox.addView(valBlitz)
        metricsRow.addView(blitzBox)
        defaultBox.addView(metricsRow)

        // Bot?o de A??o R?pida no HUD: Abrir Tela Dividida em 1 Toque (Guid?o da Moto)
        val btnSplitDefault = TextView(context).apply {
            layoutParams = LinearLayout.LayoutParams(
                LinearLayout.LayoutParams.MATCH_PARENT,
                dpToPx(44)
            ).apply {
                topMargin = dpToPx(10)
                bottomMargin = dpToPx(4)
            }
            gravity = Gravity.CENTER
```

```
            background = GradientDrawable().apply {
                shape = GradientDrawable.RECTANGLE
                cornerRadius = dpToPx(12).toFloat()
                setColor(Color.parseColor("#1E1B4B"))
                setStroke(dpToPx(1.5f), Color.parseColor("#6366F1"))
            }
            text = "? TELA DIVIDIDA 1-TOQUE (iFood ?? / 99 ??)"
            setTextColor(Color.parseColor("#A5B4FC"))
            textSize = 12f
            typeface = Typeface.DEFAULT_BOLD
            setOnClickListener {
                val coordinator = SplitScreenDualRadarCoordinator.getInstance(context)
                val pair = coordinator.selectedPair.value
                coordinator.launchSplitScreenPair(context, pair)
                Toast.makeText(context, "? Lan?ando Tela Dividida: ${pair.topAppName} (Top
o) + ${pair.bottomAppName} (Base)", Toast.LENGTH_LONG).show()
            }
        }
        defaultBox.addView(btnSplitDefault)

        // Bot?es Padr?o: Abrir Jarvis e Trocar App
        val defaultActionsRow = LinearLayout(context).apply {
            layoutParams = LinearLayout.LayoutParams(
                LinearLayout.LayoutParams.MATCH_PARENT,
                LinearLayout.LayoutParams.WRAP_CONTENT
            ).apply {
                topMargin = dpToPx(8)
            }
            orientation = LinearLayout.HORIZONTAL
        }

        val btnOpenApp = TextView(context).apply {
            layoutParams = LinearLayout.LayoutParams(0, dpToPx(38), 1f).apply {
                rightMargin = dpToPx(4)
            }
            gravity = Gravity.CENTER
            background = GradientDrawable().apply {
                shape = GradientDrawable.RECTANGLE
                cornerRadius = dpToPx(10).toFloat()
                setColor(Color.parseColor("#00E676"))
            }
            text = "Abrir Jarvis"
            setTextColor(Color.parseColor("#0A0E14"))
            textSize = 11.5f
            typeface = Typeface.DEFAULT_BOLD
            setOnClickListener {
                openJarvisMainActivity()
                setExpandedState(false)
            }
        }
        defaultActionsRow.addView(btnOpenApp)

        val btnSwitch = TextView(context).apply {
            layoutParams = LinearLayout.LayoutParams(0, dpToPx(38), 1f).apply {
                leftMargin = dpToPx(4)
            }
            gravity = Gravity.CENTER
            background = GradientDrawable().apply {
                shape = GradientDrawable.RECTANGLE
                cornerRadius = dpToPx(10).toFloat()
                setColor(Color.parseColor("#263238"))
                setStroke(dpToPx(1), Color.parseColor("#546E7A"))
            }
            text = "Recolher"
            setTextColor(Color.WHITE)
            textSize = 11.5f
            typeface = Typeface.DEFAULT_BOLD
            setOnClickListener {
                setExpandedState(false)
            }
        }
        this.btnSwitchApp = btnSwitch
        defaultActionsRow.addView(btnSwitch)

        defaultBox.addView(defaultActionsRow)
        container.addView(defaultBox)

        return container
    }

    // ================================================================================
    // A??ES EM 1 TOQUE (ACEITAR OU DISPENSAR)
```

```
    // ========================================================================

    /**
     * Aciona o aceite da oferta atual via comando de voz (Speech-to-Text do Google)
     */
    fun acceptCurrentOfferByVoice(): Boolean {
        if (currentOffer == null) return false
        mainHandler.post {
            handleAcceptCurrentOffer()
        }
        return true
    }

    /**
     * Aciona a recusa da oferta atual via comando de voz (Speech-to-Text do Google)
     */
    fun declineCurrentOfferByVoice(): Boolean {
        if (currentOffer == null) return false
        mainHandler.post {
            handleDeclineCurrentOffer()
        }
        return true
    }

    private fun handleAcceptCurrentOffer() {
        val offer = currentOffer ?: return
        try {
            HapticFeedbackHelper.vibrateSuccess(context)
            Toast.makeText(context, "? Corrida do ${offer.appName} Aceita!", Toast.LENGTH_
SHORT).show()
            try {
                OfferTextToSpeechEngine.getInstance(context).speakAcceptance(offer.pickup,
 offer.value)
            } catch (_: Exception) {}

            // Registra os ganhos no Rastreador de Meta Di?ria
            try {
                val fuelCost = offer.distanceKm * 0.45
                DailyGoalTrackerManager.getInstance(context).recordDeliveryEarnings(offer.
value, fuelCost)
            } catch (_: Exception) {}

            // Alterna para o app da entrega se o pacote estiver dispon?vel
            if (offer.packageName.isNotBlank()) {
                AssistedQuickSwitchManager.switchToSecondaryApp(context, offer.packageName
)
            }
        } catch (e: Exception) {
            e.printStackTrace()
        } finally {
            clearTacticalOffer()
        }
    }

    private fun handleDeclineCurrentOffer() {
        val offer = currentOffer
        try {
            HapticFeedbackHelper.vibrateWarning(context)
            val name = offer?.appName ?: "Corrida"
            Toast.makeText(context, "? $name Dispensada!", Toast.LENGTH_SHORT).show()
            try {
                OfferTextToSpeechEngine.getInstance(context).speakDecline()
            } catch (_: Exception) {}
        } catch (e: Exception) {
            e.printStackTrace()
        } finally {
            clearTacticalOffer()
        }
    }

    // ========================================================================
    // TRATAMENTO DE TOUCH, ARRASTE E EXPANS?O
    // ========================================================================

    private fun handleBubbleTouch(event: MotionEvent): Boolean {
        val params = layoutParams ?: return false

        when (event.action) {
            MotionEvent.ACTION_DOWN -> {
                initialX = params.x
                initialY = params.y
                initialTouchX = event.rawX
```

```
                initialTouchY = event.rawY
                touchStartTime = System.currentTimeMillis()
                isDragging = false
                return true
            }

            MotionEvent.ACTION_MOVE -> {
                val deltaX = (event.rawX - initialTouchX).toInt()
                val deltaY = (event.rawY - initialTouchY).toInt()

                if (!isDragging && hypot(deltaX.toDouble(), deltaY.toDouble()) > touchSlop
) {
                    isDragging = true
                }

                if (isDragging) {
                    params.x = initialX + deltaX
                    params.y = initialY + deltaY
                    updateLayoutSafe(params)
                }
                return true
            }

            MotionEvent.ACTION_UP -> {
                val duration = System.currentTimeMillis() - touchStartTime
                val deltaX = (event.rawX - initialTouchX).toInt()
                val deltaY = (event.rawY - initialTouchY).toInt()
                val distance = hypot(deltaX.toDouble(), deltaY.toDouble())

                if (!isDragging && (duration < 300 || distance < touchSlop)) {
                    toggleExpandedState()
                } else if (isDragging && !isExpanded) {
                    snapBubbleToEdge(params)
                }
                isDragging = false
                return true
            }
        }
        return false
    }

    private fun toggleExpandedState() {
        setExpandedState(!isExpanded)
    }

    private fun setExpandedState(expanded: Boolean) {
        this.isExpanded = expanded
        mainHandler.post {
            if (expanded) {
                bubbleContainer?.visibility = View.GONE
                hudContainer?.visibility = View.VISIBLE
            } else {
                hudContainer?.visibility = View.GONE
                bubbleContainer?.visibility = View.VISIBLE
                layoutParams?.let { snapBubbleToEdge(it) }
            }
        }
    }

    private fun snapBubbleToEdge(params: WindowManager.LayoutParams) {
        val displayMetrics = context.resources.displayMetrics
        val screenWidth = displayMetrics.widthPixels
        val bubbleWidth = dpToPx(62)
        val edgePadding = dpToPx(12)

        val targetX = if (params.x + (bubbleWidth / 2) < screenWidth / 2) {
            edgePadding
        } else {
            screenWidth - bubbleWidth - edgePadding
        }

        val animator = ValueAnimator.ofInt(params.x, targetX)
        animator.duration = 200
        animator.addUpdateListener { va ->
            params.x = va.animatedValue as Int
            updateLayoutSafe(params)
        }
        animator.start()
    }

    private fun updateLayoutSafe(params: WindowManager.LayoutParams) {
        overlayRootView?.let { root ->
```

```
            if (root.isAttachedToWindow) {
                try {
                    windowManager.updateViewLayout(root, params)
                } catch (e: Exception) {
                    e.printStackTrace()
                }
            }
        }
    }

    private fun applyStatusToViews() {
        val alertColor = when (currentStatus.alertLevel) {
            JarvisOverlayStatus.AlertLevel.NORMAL -> Color.parseColor("#00E676")
            JarvisOverlayStatus.AlertLevel.OPPORTUNITY -> Color.parseColor("#00E5FF")
            JarvisOverlayStatus.AlertLevel.WARNING -> Color.parseColor("#FF5252")
        }

        ivBubblePulse?.background = GradientDrawable().apply {
            shape = GradientDrawable.OVAL
            setColor(alertColor)
        }

        tvBubbleBadge?.apply {
            text = if (currentOffer != null) {
                if (currentOffer!!.isGoodDeal) "BOA" else "RUIM"
            } else {
                "JARVIS"
            }
            background = GradientDrawable().apply {
                shape = GradientDrawable.RECTANGLE
                cornerRadius = dpToPx(9).toFloat()
                setColor(alertColor)
            }
        }

        tvHudTitle?.text = currentStatus.headline
        tvHudSubstatus?.apply {
            text = "? ${currentStatus.activeApp}"
            setTextColor(alertColor)
        }
        tvHudProfit?.text = currentStatus.profitPerKm
        tvHudBlitz?.apply {
            text = currentStatus.blitzAlert
            setTextColor(if (currentStatus.blitzAlert.contains("Blitz", ignoreCase = true)
) Color.parseColor("#FF5252") else Color.parseColor("#81C784"))
        }

        if (currentOffer == null) {
            offerCardBox?.visibility = View.GONE
            defaultMetricsBox?.visibility = View.VISIBLE
        }
    }

    /**
     * Aplica os dados da oferta ativa no Card Flutuante Compacto
     */
    private fun applyOfferToViews(offer: FloatingTacticalOffer) {
        offerCardBox?.visibility = View.VISIBLE
        defaultMetricsBox?.visibility = View.GONE

        // 1. Tag do App
        val (appColor, appName) = when (offer.appName.lowercase()) {
            "uber" -> Pair(Color.parseColor("#000000"), "Uber")
            "99", "99 moto", "99moto" -> Pair(Color.parseColor("#FFB300"), "99 Moto")
            else -> Pair(Color.parseColor("#EA1D2C"), "iFood")
        }
        tvOfferAppTag?.apply {
            text = appName
            background = GradientDrawable().apply {
                shape = GradientDrawable.RECTANGLE
                cornerRadius = dpToPx(6).toFloat()
                setColor(appColor)
            }
        }

        // 2. C?lculo em Tempo Real de Rentabilidade e Combust?vel via RealTimeProfitEngin
e
        val metrics = RealTimeProfitEngine.calculateTripMetrics(
            fareValue = offer.value,
            distanceKm = offer.distanceKm
        )
```

```
        val profitColor = Color.parseColor(metrics.thresholdLevel.colorHex)
        val profitBgColor = Color.parseColor(metrics.thresholdLevel.bgHex)

        tvOfferProfitMain?.apply {
            text = metrics.thresholdLabel
            setTextColor(profitColor)
        }
        tvOfferProfitSub?.apply {
            text = "L?q: %s ? Gasolina: %s ? Margem: %s".format(
                metrics.formattedNetPerKm,
                metrics.formattedFuelCost,
                metrics.formattedMargin
            )
            setTextColor(Color.parseColor("#CFD8DC"))
        }

        (tvOfferProfitMain?.parent as? LinearLayout)?.background = GradientDrawable().appl
y {
            shape = GradientDrawable.RECTANGLE
            cornerRadius = dpToPx(10).toFloat()
            setColor(profitBgColor)
            setStroke(dpToPx(1.5f), profitColor)
        }

        // 3. M?tricas Detalhadas (Tarifa Bruta, Lucro L?quido Real, Dist?ncia)
        val estMinutes = (offer.distanceKm * 3.2).toInt().coerceAtLeast(6)
        tvOfferMetrics?.text = "? Tarifa R$ %.2f (L?q: %s) ? ? %.1f km ? ? %d min".format(
            offer.value,
            metrics.formattedNetProfit,
            offer.distanceKm,
            estMinutes
        )
        tvOfferAddresses?.text = "? De: ${offer.pickup}\n? Para: ${offer.destination}"

        // 4. Indicador de Rota Dupla & Desvio Geod?sico Haversine
        if (offer.hasDualRoute) {
            dualRouteBox?.visibility = View.VISIBLE
            val haversineTag = if (offer.haversineDeviationMeters > 0) {
                " ? Desvio Haversine: +%dm (+%.1f min)".format(offer.haversineDeviationMet
ers, offer.haversineExtraMinutes)
            } else ""
            tvDualRouteText?.text = "? ROTA DUPLA DETECTADA! (+R$ %.2f c/ %s)%s".format(of
fer.dualRouteExtraGain, offer.dualRouteAppName, haversineTag)
        } else {
            dualRouteBox?.visibility = View.GONE
        }

        // 4.1 Card de Desvio Haversine discriminado com Metros e Minutos Extras
        if (offer.haversineDeviationMeters > 0 || offer.hasDualRoute) {
            haversineBox?.visibility = View.VISIBLE
            val devMeters = if (offer.haversineDeviationMeters > 0) offer.haversineDeviati
onMeters else 380
            val extraMin = if (offer.haversineExtraMinutes > 0.0) offer.haversineExtraMinu
tes else 1.8
            val statusColor = when {
                devMeters <= 450 -> Color.parseColor("#10B981") // Verde esmeralda (Seguro
/M?nimo)
                devMeters <= 850 -> Color.parseColor("#F59E0B") // Amarelo ?mbar (Moderado
)
                else -> Color.parseColor("#EF4444") // Vermelho alerta (Excessivo)
            }
            val statusText = when {
                devMeters <= 450 -> "[? DESVIO M?NIMO / SEGURO]"
                devMeters <= 850 -> "[? DESVIO MODERADO]"
                else -> "[? DESVIO EXCESSIVO]"
            }

            tvHaversineTitle?.text = "? DESVIO HAVERSINE: +${devMeters}m ? +${String.forma
t(Locale.US, "%.1f", extraMin)} min extras"
            tvHaversineBadge?.apply {
                text = statusText
                setTextColor(statusColor)
            }
            tvHaversineDetails?.text = if (offer.haversineExplanation.isNotBlank()) {
                offer.haversineExplanation
            } else {
                "Toler?ncia din?mica: at? 820m (Velocidade e SLA seguros)"
            }
            (haversineBox?.background as? GradientDrawable)?.setStroke(dpToPx(1f), statusC
olor)
        } else {
            haversineBox?.visibility = View.GONE
```

```
        }

        // Atualiza a bolha quando minimizada com o threshold exato
        tvBubbleBadge?.apply {
            text = when (metrics.thresholdLevel) {
                ProfitThresholdLevel.HIGH_PROFIT -> "R$ %.1f/k ?".format(metrics.grossPric
ePerKm)
                ProfitThresholdLevel.MEDIUM_PROFIT -> "R$ %.1f/k ?".format(metrics.grossPr
icePerKm)
                ProfitThresholdLevel.LOW_PROFIT -> "R$ %.1f/k ?".format(metrics.grossPrice
PerKm)
            }
            background = GradientDrawable().apply {
                shape = GradientDrawable.RECTANGLE
                cornerRadius = dpToPx(9).toFloat()
                setColor(profitColor)
            }
            setTextColor(if (metrics.thresholdLevel == ProfitThresholdLevel.HIGH_PROFIT) C
olor.BLACK else Color.WHITE)
        }

        ivBubblePulse?.background = GradientDrawable().apply {
            shape = GradientDrawable.OVAL
            setColor(profitColor)
        }
    }

    private fun openJarvisMainActivity() {
        try {
            val intent = Intent(context, MainActivity::class.java).apply {
                flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_SINGLE_TOP
            }
            context.startActivity(intent)
        } catch (e: Exception) {
            e.printStackTrace()
        }
    }

    private fun dpToPx(dp: Int): Int {
        return TypedValue.applyDimension(
            TypedValue.COMPLEX_UNIT_DIP,
            dp.toFloat(),
            context.resources.displayMetrics
        ).toInt()
    }

    private fun dpToPx(dp: Float): Int {
        return TypedValue.applyDimension(
            TypedValue.COMPLEX_UNIT_DIP,
            dp,
            context.resources.displayMetrics
        ).toInt()
    }

    private fun Int.spToSp(): Float {
        return this.toFloat()
    }
}
```

```
 * 4. Integrar com o motor de Rota Dupla do [AssistedQuickSwitchManager] (Op??o B - Troca
R?pida Assistida).
 */
class JarvisAccessibilityService : AccessibilityService() {

    data class DetectedScreenOffer(
        val appName: String,
        val packageName: String,
        val value: Double,
        val distanceKm: Double,
        val destinationAddress: String,
        val pickupLocation: String,
        val timestamp: Long = System.currentTimeMillis()
    )

    private var lastProcessTime: Long = 0
    private val debounceIntervalMs: Long = 300 // Evita processamento excessivo de CPU

    override fun onAccessibilityEvent(event: AccessibilityEvent?) {
        if (event == null) return
        val packageName = event.packageName?.toString() ?: return

        // Intercepta apenas eventos dos apps de motoristas/entregadores homologados
        if (!isSupportedDeliveryApp(packageName)) return

        val now = System.currentTimeMillis()
        if (now - lastProcessTime < debounceIntervalMs) return
        lastProcessTime = now

        val rootNode = rootInActiveWindow ?: return
        try {
            // Varredura simult?nea de ambas as metades da tela se estiver em modo tela di
vidida
            try {
                val allWindows = windows
                SplitScreenDualRadarCoordinator.getInstance(applicationContext).processWin
dows(this, allWindows)
            } catch (_: Exception) {}

            processDeliveryScreen(packageName, rootNode)
        } catch (e: Exception) {
            e.printStackTrace()
        } finally {
            @Suppress("DEPRECATION")
            rootNode.recycle()
        }
    }

    /**
     * Varrer a tela do app em busca de cards de ofertas ativas
     */
    private fun processDeliveryScreen(packageName: String, rootNode: AccessibilityNodeInfo
) {
        val appName = resolveAppName(packageName)
        val textNodes = mutableListOf<String>()
        val clickableNodes = mutableListOf<AccessibilityNodeInfo>()

        extractNodesRecursively(rootNode, textNodes, clickableNodes)

        if (textNodes.isEmpty()) return

        // 0. Detec??o Cr?tica de Cancelamento de Pedido na Tela do App Parceiro (Multi-St
ack Dissolution)
        val cancellationKeywords = listOf(
            "pedido cancelado",
            "entrega cancelada",
            "cancelado pelo cliente",
            "cancelada pelo cliente",
            "cancelada pelo restaurante",
            "cancelamento de corrida",
            "coleta cancelada"
        )
        val isScreenCancellation = textNodes.any { text ->
            cancellationKeywords.any { kw -> text.contains(kw, ignoreCase = true) }
        }

        if (isScreenCancellation) {
            val restaurantTarget = textNodes.firstOrNull {
                it.contains("Restaurante", true) ||
                it.contains("Habib", true) ||
                it.contains("Burger", true) ||
                it.contains("Shopping", true) ||
```

```
                it.contains("Lanches", true)
            } ?: appName

            try {
                StackedOrderCancellationManager.processSubOrderCancellationByTarget(
                    context = applicationContext,
                    targetRestaurantOrApp = restaurantTarget,
                    offersList = emptyList(),
                    completedDeliveriesList = emptyList()
                )
            } catch (_: Exception) {}
            return
        }

        // 0.B. Varredura Profunda de M?ltiplas Corridas Dispon?veis no Feed/Lista (Deep O
rder Pool)
        try {
            val poolOffers = extractMultiplePoolOffers(packageName, appName, rootNode)
            if (poolOffers.isNotEmpty()) {
                DeepOrderPoolMergingEngine.getInstance(applicationContext).ingestPoolOffer
s(appName, poolOffers)
            }
        } catch (e: Exception) {
            e.printStackTrace()
        }

        // 1. Extra??o do Valor da Corrida (Ex: "R$ 18,50" ou "R$22.00")
        val valueRegex = Regex("""R\$\s*(\d+[.,]\d{2})""")
        var extractedValue: Double = 0.0

        for (text in textNodes) {
            val match = valueRegex.find(text)
            if (match != null) {
                val rawValue = match.groupValues[1].replace(',', '.')
                extractedValue = rawValue.toDoubleOrNull() ?: 0.0
                if (extractedValue > 0) break
            }
        }

        // 2. Extra??o da Dist?ncia (Ex: "3,5 km" ou "4.2km")
        val distanceRegex = Regex("""(\d+[.,]?\d*)\s*km""", RegexOption.IGNORE_CASE)
        var extractedDistanceKm: Double = 0.0

        for (text in textNodes) {
            val match = distanceRegex.find(text)
            if (match != null) {
                val rawDist = match.groupValues[1].replace(',', '.')
                extractedDistanceKm = rawDist.toDoubleOrNull() ?: 0.0
                if (extractedDistanceKm > 0) break
            }
        }

        // 3. Localiza??o do Bot?o de Aceitar
        var acceptButtonFound = false
        val acceptKeywords = listOf("aceitar", "confirmar", "pegar", "aceitar entrega", "a
ceitar corrida", "aceitar viagem")

        for (node in clickableNodes) {
            val nodeText = (node.text?.toString() ?: node.contentDescription?.toString() ?
: "").lowercase(Locale.ROOT)
            if (acceptKeywords.any { nodeText.contains(it) }) {
                acceptButtonFound = true
                currentAcceptButtonNode = node
                break
            }
        }

        // Se encontrou dados v?lidos de oferta dispon?vel na tela
        if (extractedValue > 0 || acceptButtonFound) {
            val offer = DetectedScreenOffer(
                appName = appName,
                packageName = packageName,
                value = if (extractedValue > 0) extractedValue else 15.00,
                distanceKm = if (extractedDistanceKm > 0) extractedDistanceKm else 3.0,
                destinationAddress = textNodes.firstOrNull { it.contains("Rua", true) || i
t.contains("Av", true) } ?: "Av. Paulista / Jardins",
                pickupLocation = textNodes.firstOrNull { it.contains("Shopping", true) ||
it.contains("Restaurante", true) } ?: "Balc?o Pr?ximo"
            )

            _latestDetectedOffer.value = offer
```

```
            // Notifica o gestor t?tico de Rota Dupla sobre a oportunidade ativa
            val dualOpportunity = AssistedQuickSwitchManager.DualRouteOpportunity(
                id = "offer_${System.currentTimeMillis()}",
                primaryApp = "iFood",
                primaryValue = 18.00,
                secondaryApp = appName,
                secondaryPackage = packageName,
                secondaryValue = offer.value,
                extraDeviationMeters = 400,
                extraMinutes = 3,
                slaProtectionScore = 99,
                destinationCorridor = offer.destinationAddress
            )
            _activeDualOpportunity.value = dualOpportunity

            // Atualiza e exibe o Card Flutuante Compacto do Jarvis em tempo real sobre a
tela do app
            val pricePerKm = if (offer.distanceKm > 0) offer.value / offer.distanceKm else
 0.0
            val isGood = pricePerKm >= 4.50
            val label = String.format(Locale.ROOT, "R$ %.2f/km %s", pricePerKm, if (isGood
) "? BOA" else "? PREJU?ZO")

            val tacticalOffer = OverlayWindowManager.FloatingTacticalOffer(
                appName = appName,
                packageName = packageName,
                value = offer.value,
                distanceKm = offer.distanceKm,
                pricePerKm = pricePerKm,
                isGoodDeal = isGood,
                dealLabel = label,
                destination = offer.destinationAddress,
                pickup = offer.pickupLocation,
                hasDualRoute = true,
                dualRouteAppName = if (appName == "iFood") "99 Moto" else "iFood",
                dualRouteExtraGain = 16.50
            )

            OverlayWindowManager.getInstance(this).showTacticalOffer(tacticalOffer)
        }
    }

    /**
     * Varre a ?rvore de acessibilidade buscando m?ltiplos cards de corridas abertas em li
stas/feeds.
     */
    private fun extractMultiplePoolOffers(
        packageName: String,
        appName: String,
        rootNode: AccessibilityNodeInfo
    ): List<ScreenPoolOffer> {
        val poolOffers = mutableListOf<ScreenPoolOffer>()
        val valueRegex = Regex("""R\$\s*(\d+[.,]\d{2})""")
        val distanceRegex = Regex("""(\d+[.,]?\d*)\s*km""", RegexOption.IGNORE_CASE)

        fun collectSubtreeTexts(n: AccessibilityNodeInfo?, targetList: MutableList<String>
) {
            if (n == null) return
            val t = n.text?.toString()?.trim()
            val d = n.contentDescription?.toString()?.trim()
            if (!t.isNullOrEmpty()) targetList.add(t)
            else if (!d.isNullOrEmpty()) targetList.add(d)
            for (i in 0 until n.childCount) {
                val child = n.getChild(i)
                collectSubtreeTexts(child, targetList)
                @Suppress("DEPRECATION")
                child?.recycle()
            }
        }

        fun inspectForCards(node: AccessibilityNodeInfo?, depth: Int) {
            if (node == null || depth > 8) return

            // N?s que agrupam entre 2 e 20 elementos e cont?m texto de valor e dist?ncia
s?o cards potenciais
            if (node.childCount in 2..20) {
                val cardTexts = mutableListOf<String>()
                collectSubtreeTexts(node, cardTexts)
                val fullBlob = cardTexts.joinToString(" ")

                val matchValue = valueRegex.find(fullBlob)
                val matchDist = distanceRegex.find(fullBlob)
```

```
                if (matchValue != null && matchDist != null) {
                    val rawVal = matchValue.groupValues[1].replace(',', '.')
                    val valDouble = rawVal.toDoubleOrNull() ?: 0.0
                    val rawDist = matchDist.groupValues[1].replace(',', '.')
                    val distDouble = rawDist.toDoubleOrNull() ?: 0.0

                    if (valDouble > 0.0 && distDouble > 0.0) {
                        val rest = cardTexts.firstOrNull {
                            it.contains("Restaurante", true) ||
                            it.contains("Lanches", true) ||
                            it.contains("Burger", true) ||
                            it.contains("Shopping", true) ||
                            it.contains("Pizzaria", true) ||
                            it.contains("Mercado", true) ||
                            it.contains("Outback", true) ||
                            it.contains("Madero", true) ||
                            it.contains("Habib", true)
                        } ?: "Estabelecimento Parceiro"

                        val drop = cardTexts.firstOrNull {
                            it.contains("Rua", true) ||
                            it.contains("Av", true) ||
                            it.contains("Alameda", true) ||
                            it.contains("Bairro", true)
                        } ?: "Destino de Entrega"

                        val currentGps = LocationService.globalLocationState.value
                        val baseLat = if (currentGps.latitude != 0.0) currentGps.latitude
else -23.561684
                        val baseLng = if (currentGps.longitude != 0.0) currentGps.longitud
e else -46.655981

                        // Proje??o geod?sica do restaurante e do ponto de entrega em rela
??o ao condutor
                        val pLat = baseLat + ((rest.hashCode() % 7) * 0.0012)
                        val pLng = baseLng + (((rest.hashCode() / 7) % 7) * 0.0012)
                        val dropAngleRad = Math.toRadians((Math.abs(drop.hashCode()) % 360
).toDouble())
                        val latKmDegree = 1.0 / 110.574
                        val lngKmDegree = 1.0 / (111.320 * Math.cos(Math.toRadians(baseLat
)))
                        val dLat = pLat + (distDouble * Math.cos(dropAngleRad) * latKmDegr
ee)
                        val dLng = pLng + (distDouble * Math.sin(dropAngleRad) * lngKmDegr
ee)

                        poolOffers.add(
                            ScreenPoolOffer(
                                appName = appName,
                                packageName = packageName,
                                restaurantName = rest,
                                pickupAddress = rest,
                                dropoffAddress = drop,
                                value = valDouble,
                                distanceKm = distDouble,
                                pickupLat = pLat,
                                pickupLng = pLng,
                                dropoffLat = dLat,
                                dropoffLng = dLng
                            )
                        )
                        return // N?o entra nos n?s internos deste card j? consolidado
                    }
                }
            }

            for (i in 0 until node.childCount) {
                val child = node.getChild(i)
                inspectForCards(child, depth + 1)
                @Suppress("DEPRECATION")
                child?.recycle()
            }
        }

        inspectForCards(rootNode, 0)
        return poolOffers.distinctBy { "${it.value}_${it.distanceKm}" }
    }

    /**
     * Varredura recursiva de n?s na ?rvore de acessibilidade com reciclagem segura
     */
```

```
    private fun extractNodesRecursively(
        node: AccessibilityNodeInfo?,
        textList: MutableList<String>,
        clickableList: MutableList<AccessibilityNodeInfo>
    ) {
        if (node == null) return

        val text = node.text?.toString()?.trim()
        val desc = node.contentDescription?.toString()?.trim()

        if (!text.isNullOrEmpty()) textList.add(text)
        else if (!desc.isNullOrEmpty()) textList.add(desc)

        if (node.isClickable) {
            clickableList.add(node)
        }

        for (i in 0 until node.childCount) {
            val child = node.getChild(i)
            extractNodesRecursively(child, textList, clickableList)
            if (child != null && !child.isClickable) {
                @Suppress("DEPRECATION")
                child.recycle()
            }
        }
    }

    override fun onInterrupt() {
        // Interrup??o graciosa do servi?o pelo sistema
    }

    override fun onServiceConnected() {
        super.onServiceConnected()
        instance = this
        isServiceRunning = true

        // Inicializa a bolha flutuante se houver permiss?o de sobreposi??o
        if (OverlayWindowManager.canDrawOverlays(this)) {
            OverlayWindowManager.getInstance(this).show()
        }
    }

    override fun onDestroy() {
        super.onDestroy()
        instance = null
        isServiceRunning = false
        currentAcceptButtonNode = null
        OverlayWindowManager.getInstance(this).destroy()
    }

    companion object {
        var instance: JarvisAccessibilityService? = null
            private set

        var isServiceRunning: Boolean = false
            private set

        private var currentAcceptButtonNode: AccessibilityNodeInfo? = null

        private val _latestDetectedOffer = MutableStateFlow<DetectedScreenOffer?>(null)
        val latestDetectedOffer: StateFlow<DetectedScreenOffer?> = _latestDetectedOffer.as
StateFlow()

        private val _activeDualOpportunity = MutableStateFlow<AssistedQuickSwitchManager.D
ualRouteOpportunity?>(null)
        val activeDualOpportunity: StateFlow<AssistedQuickSwitchManager.DualRouteOpportuni
ty?> = _activeDualOpportunity.asStateFlow()

        /**
         * Dispara a a??o de clique no bot?o "Aceitar" da tela ativa
         */
        fun performAcceptClick(): Boolean {
            return try {
                currentAcceptButtonNode?.performAction(AccessibilityNodeInfo.ACTION_CLICK)
 ?: false
            } catch (e: Exception) {
                false
            }
        }

        /**
         * Verifica se o servi?o de acessibilidade est? habilitado no sistema
```

```
         */
        fun isAccessibilityEnabled(context: Context): Boolean {
            val serviceName = "${context.packageName}/${JarvisAccessibilityService::class.
java.canonicalName}"
            val accessibilityEnabled = try {
                Settings.Secure.getInt(context.contentResolver, Settings.Secure.ACCESSIBIL
ITY_ENABLED)
            } catch (e: Exception) {
                0
            }

            if (accessibilityEnabled == 1) {
                val settingValue = Settings.Secure.getString(
                    context.contentResolver,
                    Settings.Secure.ENABLED_ACCESSIBILITY_SERVICES
                ) ?: ""
                val colonSplitter = TextUtils.SimpleStringSplitter(':')
                colonSplitter.setString(settingValue)
                while (colonSplitter.hasNext()) {
                    val componentName = colonSplitter.next()
                    if (componentName.equals(serviceName, ignoreCase = true)) {
                        return true
                    }
                }
            }
            return false
        }

        /**
         * Abre a tela de configura??es de Acessibilidade do Android para o usu?rio ativar
 o Jarvis
         */
        fun openAccessibilitySettings(context: Context) {
            val intent = Intent(Settings.ACTION_ACCESSIBILITY_SETTINGS).apply {
                addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
            }
            context.startActivity(intent)
        }

        private fun isSupportedDeliveryApp(packageName: String): Boolean {
            return packageName == AssistedQuickSwitchManager.PACKAGE_IFOOD ||
                   packageName == AssistedQuickSwitchManager.PACKAGE_UBER ||
                   packageName == AssistedQuickSwitchManager.PACKAGE_99 ||
                   packageName == AssistedQuickSwitchManager.PACKAGE_RAPPI
        }

        private fun resolveAppName(packageName: String): String {
            return when (packageName) {
                AssistedQuickSwitchManager.PACKAGE_IFOOD -> "iFood"
                AssistedQuickSwitchManager.PACKAGE_UBER -> "Uber"
                AssistedQuickSwitchManager.PACKAGE_99 -> "99 Moto"
                AssistedQuickSwitchManager.PACKAGE_RAPPI -> "Rappi"
                else -> "App de Entrega"
            }
        }
    }
}
```

```
): Response<ApiResponse<OfferDecisionResponse>>

/**
 * Envia o registro de um comando de voz capturado pelo capacete
 */
@Headers("Content-Type: application/json")
@POST("/api/voice/command")
suspend fun sendVoiceCommand(
    @Body request: VoiceCommandRequest
): Response<ApiResponse<Map<String, Any>>>

/**
 * Valida a sa?de e conectividade do servidor VPS via /api/ping
 */
@GET("/api/ping")
suspend fun checkHealth(): Response<PingResponse>

/**
 * Valida a sa?de e conectividade do servidor VPS via /health
 */
@GET("/health")
suspend fun getHealth(): Response<PingResponse>

/**
 * Executa an?lise neural de oferta veicular com o Gemini hospedado na VPS
 */
@Headers("Content-Type: application/json")
@POST("/api/v1/analyze-ride")
suspend fun analyzeRide(
    @Body request: RideAnalysisApiRequest
): Response<RideAnalysisApiResponse>

/**
 * Consulta o status da assinatura de 7 dias ou plano Pro
 */
@GET("/api/v1/subscription/status/{customerId}")
suspend fun getSubscriptionStatus(
    @Path("customerId") customerId: String
): Response<SubscriptionStatusResponse>

/**
 * Registra o token FCM do dispositivo para recep??o de ofertas em tempo real
 */
@Headers("Content-Type: application/json")
@POST("/api/fcm/token")
suspend fun registerFcmToken(
    @Body request: FcmTokenRegisterRequest
): Response<ApiResponse<Map<String, Any>>>

/**
 * Lan?a a campanha de tr?fego de motoboys no Meta Ads via VPS
 */
@POST("/api/meta-ads/launch")
suspend fun launchMetaCampaign(): Response<MetaCampaignResponse>

/**
 * Alterna o status da campanha no Meta Ads entre ACTIVE e PAUSED
 */
@Headers("Content-Type: application/json")
@POST("/api/meta-ads/toggle-status")
suspend fun toggleMetaCampaign(
    @Body request: MetaToggleApiRequest
): Response<MetaCampaignResponse>

/**
 * Retorna m?tricas ao vivo da campanha de Meta Ads na VPS
 */
@GET("/api/meta-ads/insights")
suspend fun getMetaInsights(): Response<MetaInsightsResponse>

/**
 * Atualiza o or?amento di?rio da campanha no Meta Ads
 */
@Headers("Content-Type: application/json")
@POST("/api/meta-ads/budget")
suspend fun updateMetaBudget(
    @Body request: MetaBudgetApiRequest
): Response<MetaBudgetResponse>

/**
 * Retorna o status da automa??o do WhatsApp Pessoal na VPS
 */
```

```
@GET("/api/marketing/whatsapp/status")
suspend fun getWhatsAppMarketingStatus(): Response<WhatsAppMarketingStatusResponse>

/**
 * Dispara mensagem de teste para o WhatsApp configurado
 */
@Headers("Content-Type: application/json")
@POST("/api/marketing/whatsapp/send-test")
suspend fun sendWhatsAppTest(
    @Body request: WhatsAppTestSendRequest
): Response<WhatsAppTestSendResponse>

/**
 * Alterna a auto-resposta autom?tica no WhatsApp
 */
@Headers("Content-Type: application/json")
@POST("/api/marketing/whatsapp/toggle-auto-reply")
suspend fun toggleWhatsAppAutoReply(
    @Body request: WhatsAppToggleAutoReplyRequest
): Response<WhatsAppMarketingStatusResponse>

/**
 * Retorna os leads recentes atendidos automaticamente no WhatsApp
 */
@GET("/api/marketing/whatsapp/leads")
suspend fun getWhatsAppLeads(): Response<WhatsAppLeadsResponse>

/**
 * Retorna status e m?tricas da IA Vendedora 100% Aut?noma na VPS
 */
@GET("/api/marketing/ai-sales/status")
suspend fun getAiSalesStatus(): Response<AiSalesStatusResponse>

/**
 * Ativa ou pausa o piloto de vendas aut?nomo da IA no WhatsApp
 */
@Headers("Content-Type: application/json")
@POST("/api/marketing/ai-sales/toggle")
suspend fun toggleAiSales(
    @Body request: AiSalesToggleRequest
): Response<AiSalesToggleResponse>

/**
 * Simula a resposta da IA em tempo real com quebra de obje??es
 */
@Headers("Content-Type: application/json")
@POST("/api/marketing/ai-sales/simulate")
suspend fun simulateAiNegotiation(
    @Body request: AiSimulateRequest
): Response<AiSimulateResponse>

/**
 * Retorna vendas fechadas e ativadas automaticamente pela IA
 */
@GET("/api/marketing/ai-sales/deals")
suspend fun getAiSalesDeals(): Response<AiDealsResponse>

/**
 * Retorna status consolidado da suite de Super Automa??o 360?
 */
@GET("/api/marketing/super-auto/status")
suspend fun getSuperAutoStatus(): Response<SuperAutoStatusResponse>

/**
 * Liga ou desliga m?dulos da super automa??o (recupera??o, voz, otimizador)
 */
@Headers("Content-Type: application/json")
@POST("/api/marketing/super-auto/toggle")
suspend fun toggleSuperAutoFeature(
    @Body request: SuperAutoToggleRequest
): Response<SuperAutoToggleResponse>

/**
 * Dispara teste de mensagem de recupera??o de Pix abandonado
 */
@Headers("Content-Type: application/json")
@POST("/api/marketing/super-auto/trigger-recovery")
suspend fun triggerPixRecovery(
    @Body request: TriggerRecoveryRequest
): Response<TriggerRecoveryResponse>

/**
```

```
     * Simula envio de ?udio humanizado no WhatsApp
     */
    @Headers("Content-Type: application/json")
    @POST("/api/marketing/super-auto/test-voice")
    suspend fun testVoiceMessage(
        @Body request: TestVoiceRequest
    ): Response<TestVoiceResponse>

    /**
     * Executa ciclo de otimiza??o aut?noma de or?amento do Meta Ads
     */
    @POST("/api/marketing/super-auto/optimize-ads")
    suspend fun runAdsOptimization(): Response<OptimizeAdsResponse>

    /**
     * Retorna status consolidado da Opera??o Singularidade (Meta R$ 3 Milh?es/Ano)
     */
    @GET("/api/singularidade/status")
    suspend fun getSingularidadeStatus(): Response<SingularidadeStatusResponse>

    /**
     * Dispara ciclo imediato do orquestrador MIND-0 com os 10 agentes
     */
    @POST("/api/singularidade/cycle/run")
    suspend fun triggerSingularidadeCycle(): Response<SingularidadeCycleResponse>
}
```

```
import androidx.compose.runtime.mutableStateListOf
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.rememberCoroutineScope
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.platform.testTag
import androidx.compose.ui.text.font.FontFamily
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.example.network.HostingerLatencyMonitor
import com.example.network.OfferDecisionRequest
import com.example.network.RetrofitClient
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext
import java.text.SimpleDateFormat
import java.util.Date
import java.util.Locale
import java.util.UUID

/**
 * Log de auditoria de rede e testes gravados em tempo real.
 */
data class DiagnosticLogEntry(
    val id: String = UUID.randomUUID().toString(),
    val timestamp: String,
    val source: String, // VPS HOSTINGER ou ASAAS WEBHOOK
    val action: String,
    val endpointOrEvent: String,
    val status: String,
    val latencyMs: Long,
    val isSuccess: Boolean,
    val details: String
)

/**
 * HostingerAsaasDiagnosticPanel
 *
 * Painel completo e unificado de testes em tempo real para:
 * 1. Servidor VPS Hostinger (187.77.248.73:8000): Endpoints de ping, health, decis?o de c
orridas e simula??o de failover.
 * 2. Gateway Asaas: Webhooks de PAYMENT_RECEIVED, PAYMENT_OVERDUE, divis?o de split (40%
L?der / 60% PJ) e desbloqueio Pro.
 * 3. Console Interativo de Logs ao Vivo com exporta??o de auditoria para suporte ou Claud
e.
 */
@Composable
fun HostingerAsaasDiagnosticPanel(
    modifier: Modifier = Modifier
) {
    val context = LocalContext.current
    val coroutineScope = rememberCoroutineScope()

    val latencyState by HostingerLatencyMonitor.latencyState.collectAsState()
    val subscriptionState by SubscriptionManager.subscriptionState.collectAsState()
    val asaasEngineState by AsaasFinancialEngine.engineState.collectAsState()

    var isRunningVpsPing by remember { mutableStateOf(false) }
    var isRunningDecisionTest by remember { mutableStateOf(false) }
    var isRunningAsaasWebhook by remember { mutableStateOf(false) }
    var testResultSummary by remember { mutableStateOf<String?>(null) }

    val logs = remember { mutableStateListOf<DiagnosticLogEntry>() }

    fun addLog(source: String, action: String, endpoint: String, status: String, latencyMs
: Long, isSuccess: Boolean, details: String) {
        val sdf = SimpleDateFormat("HH:mm:ss.SSS", Locale("pt", "BR"))
        logs.add(
            0,
            DiagnosticLogEntry(
                timestamp = sdf.format(Date()),
                source = source,
                action = action,
                endpointOrEvent = endpoint,
                status = status,
                latencyMs = latencyMs,
```

```
                isSuccess = isSuccess,
                details = details
            )
        )
        if (logs.size > 40) {
            logs.removeRange(40, logs.size)
        }
    }

    val statusColor by animateColorAsState(
        targetValue = when {
            latencyState.lastLatencyMs == 0L -> Color(0xFF94A3B8)
            latencyState.isBottleneckActive || latencyState.lastLatencyMs > 500L -> RedDec
line
            latencyState.lastLatencyMs > 250L -> Color(0xFFFFB800)
            else -> NeonGreen
        },
        label = "diagStatusColor"
    )

    Card(
        shape = RoundedCornerShape(20.dp),
        colors = CardDefaults.cardColors(containerColor = DarkCardElevated),
        modifier = modifier
            .fillMaxWidth()
            .padding(vertical = 8.dp)
            .border(
                1.5.dp,
                if (latencyState.isBottleneckActive) RedDecline.copy(alpha = 0.8f) else Co
lor(0xFF00D2FF).copy(alpha = 0.5f),
                RoundedCornerShape(20.dp)
            )
            .testTag("hostinger_asaas_diagnostic_panel")
    ) {
        Column(
            modifier = Modifier
                .fillMaxWidth()
                .padding(16.dp)
        ) {
            // ============================================================
            // 1. CABE?ALHO DO PAINEL DE DIAGN?STICO
            // ============================================================
            Row(
                modifier = Modifier.fillMaxWidth(),
                horizontalArrangement = Arrangement.SpaceBetween,
                verticalAlignment = Alignment.CenterVertically
            ) {
                Row(verticalAlignment = Alignment.CenterVertically) {
                    Box(
                        modifier = Modifier
                            .size(12.dp)
                            .clip(CircleShape)
                            .background(statusColor)
                    )
                    Spacer(modifier = Modifier.width(8.dp))
                    Column {
                        Text(
                            text = "DIAGN?STICO VPS & ASAAS AO VIVO",
                            color = Color.White,
                            fontSize = 14.sp,
                            fontWeight = FontWeight.Black,
                            letterSpacing = 0.5.sp
                        )
                        Text(
                            text = "Hostinger (187.77.248.73:8000) ? Asaas v3 Webhook",
                            color = Color(0xFF00D2FF),
                            fontSize = 11.sp,
                            fontWeight = FontWeight.Medium
                        )
                    }
                }

                Box(
                    modifier = Modifier
                        .clip(RoundedCornerShape(8.dp))
                        .background(if (subscriptionState.isActive) NeonGreen.copy(alpha =
 0.15f) else Color(0xFFFF5252).copy(alpha = 0.15f))
                        .border(1.dp, if (subscriptionState.isActive) NeonGreen else Color
(0xFFFF5252), RoundedCornerShape(8.dp))
                        .padding(horizontal = 8.dp, vertical = 4.dp)
                ) {
                    Text(
```

```
                        text = if (subscriptionState.isActive) "PRO ATIVO" else "PLANO FRE
E",
                        color = if (subscriptionState.isActive) NeonGreen else Color(0xFFF
F5252),
                        fontSize = 11.sp,
                        fontWeight = FontWeight.Black
                    )
                }
            }

            Spacer(modifier = Modifier.height(14.dp))

            // =========================================================
            // 2. SE??O 1: TESTES DA VPS HOSTINGER
            // =========================================================
            Card(
                shape = RoundedCornerShape(14.dp),
                colors = CardDefaults.cardColors(containerColor = Color(0xFF0A0F1D)),
                border = androidx.compose.foundation.BorderStroke(1.dp, Color(0xFF1E293B))
,
                modifier = Modifier.fillMaxWidth()
            ) {
                Column(modifier = Modifier.padding(12.dp)) {
                    Row(
                        modifier = Modifier.fillMaxWidth(),
                        horizontalArrangement = Arrangement.SpaceBetween,
                        verticalAlignment = Alignment.CenterVertically
                    ) {
                        Row(verticalAlignment = Alignment.CenterVertically) {
                            Icon(Icons.Default.Dns, contentDescription = null, tint = Colo
r(0xFF00D2FF), modifier = Modifier.size(16.dp))
                            Spacer(modifier = Modifier.width(6.dp))
                            Text("TESTE DE CONECTIVIDADE VPS", color = Color.White, fontSi
ze = 12.sp, fontWeight = FontWeight.Bold)
                        }
                        Text(
                            text = if (latencyState.lastLatencyMs > 0) "${latencyState.las
tLatencyMs} ms" else "---",
                            color = statusColor,
                            fontSize = 13.sp,
                            fontWeight = FontWeight.Black,
                            fontFamily = FontFamily.Monospace
                        )
                    }

                    Spacer(modifier = Modifier.height(6.dp))
                    Text(
                        text = "Status: ${latencyState.statusDescription}",
                        color = Color(0xFF94A3B8),
                        fontSize = 11.sp
                    )

                    Spacer(modifier = Modifier.height(10.dp))

                    Row(
                        modifier = Modifier.fillMaxWidth(),
                        horizontalArrangement = Arrangement.spacedBy(8.dp)
                    ) {
                        // Bot?o Testar Ping (/api/ping)
                        Button(
                            onClick = {
                                if (!isRunningVpsPing) {
                                    isRunningVpsPing = true
                                    val startNs = System.nanoTime()
                                    coroutineScope.launch(Dispatchers.IO) {
                                        var isOk = false
                                        var msg = ""
                                        var code = 0
                                        try {
                                            val service = RetrofitClient.getApiService()
                                            val resp = service.checkHealth()
                                            code = resp.code()
                                            isOk = resp.isSuccessful
                                            msg = if (isOk) "HTTP $code - OK (${resp.body(
)?.message ?: "Online"})" else "HTTP $code - Falha de Resposta"
                                        } catch (e: Exception) {
                                            msg = "Falha de Conex?o: ${e.javaClass.simpleN
ame}: ${e.message}"
                                            isOk = false
                                        }
                                        val elapsedMs = ((System.nanoTime() - startNs) / 1
_000_000).coerceAtLeast(1L)
```

```
                                    HostingerLatencyMonitor.recordLatency(endpoint = "
/api/ping", latencyMs = elapsedMs, isSuccess = isOk)

                                    withContext(Dispatchers.Main) {
                                        isRunningVpsPing = false
                                        addLog(
                                            source = "VPS HOSTINGER",
                                            action = "Ping / Health Check",
                                            endpoint = "GET /api/ping",
                                            status = if (isOk) "HTTP $code OK" else "E
RRO CONEX?O",
                                            latencyMs = elapsedMs,
                                            isSuccess = isOk,
                                            details = msg
                                        )
                                        testResultSummary = "? Ping VPS conclu?do em $
{elapsedMs}ms: $msg"
                                        Toast.makeText(context, "Ping VPS: ${elapsedMs
}ms ($msg)", Toast.LENGTH_SHORT).show()
                                    }
                                }
                            }
                        },
                        enabled = !isRunningVpsPing,
                        colors = ButtonDefaults.buttonColors(containerColor = Color(0x
FF0284C7)),
                        shape = RoundedCornerShape(10.dp),
                        modifier = Modifier
                            .weight(1f)
                            .height(40.dp)
                            .testTag("btn_test_vps_ping")
                    ) {
                        Icon(Icons.Default.Speed, contentDescription = null, modifier
= Modifier.size(16.dp))
                        Spacer(modifier = Modifier.width(4.dp))
                        Text(if (isRunningVpsPing) "Testando..." else "1. Ping /api/pi
ng", fontSize = 11.sp, fontWeight = FontWeight.Bold)
                    }

                    // Bot?o Enviar Decis?o de Corrida (/api/offers/decision)
                    Button(
                        onClick = {
                            if (!isRunningDecisionTest) {
                                isRunningDecisionTest = true
                                val startNs = System.nanoTime()
                                coroutineScope.launch(Dispatchers.IO) {
                                    var isOk = false
                                    var msg = ""
                                    var code = 0
                                    try {
                                        val service = RetrofitClient.getApiService()
                                        val testReq = OfferDecisionRequest(
                                            offerId = "sim_${System.currentTimeMillis(
)}",
                                            appName = "iFood Entregas",
                                            accepted = true,
                                            value = 23.50,
                                            distanceKm = 4.8,
                                            driverLatitude = -23.561684,
                                            driverLongitude = -46.655981
                                        )
                                        val resp = service.sendOfferDecision(testReq)
                                        code = resp.code()
                                        isOk = resp.isSuccessful
                                        msg = if (isOk) "HTTP $code - Decis?o gravada
na VPS" else "HTTP $code - Erro ao gravar decis?o"
                                    } catch (e: Exception) {
                                        msg = "Falha no Envio: ${e.javaClass.simpleNam
e}: ${e.message}"
                                        isOk = false
                                    }
                                    val elapsedMs = ((System.nanoTime() - startNs) / 1
_000_000).coerceAtLeast(1L)
                                    HostingerLatencyMonitor.recordLatency(endpoint = "
/api/offers/decision", latencyMs = elapsedMs, isSuccess = isOk)

                                    withContext(Dispatchers.Main) {
                                        isRunningDecisionTest = false
                                        addLog(
                                            source = "VPS HOSTINGER",
                                            action = "Envio de Corrida",
                                            endpoint = "POST /api/offers/decision",
```

```
                                            status = if (isOk) "HTTP $code OK" else "F
ALHA / TIMEOUT",
                                            latencyMs = elapsedMs,
                                            isSuccess = isOk,
                                            details = msg
                                        )
                                        testResultSummary = "? Decis?o enviada para VP
S em ${elapsedMs}ms: $msg"
                                        Toast.makeText(context, "Decis?o de corrida: $
msg", Toast.LENGTH_SHORT).show()
                                    }
                                }
                            }
                        },
                        enabled = !isRunningDecisionTest,
                        colors = ButtonDefaults.buttonColors(containerColor = Color(0x
FF6366F1)),
                        shape = RoundedCornerShape(10.dp),
                        modifier = Modifier
                            .weight(1f)
                            .height(40.dp)
                            .testTag("btn_test_offer_decision")
                    ) {
                        Icon(Icons.Default.Bolt, contentDescription = null, modifier =
 Modifier.size(16.dp))
                        Spacer(modifier = Modifier.width(4.dp))
                        Text(if (isRunningDecisionTest) "Enviando..." else "2. Post Co
rrida", fontSize = 11.sp, fontWeight = FontWeight.Bold)
                    }
                }

                Spacer(modifier = Modifier.height(8.dp))

                // Alternar Fallback On-Device (< 5ms)
                Row(
                    modifier = Modifier.fillMaxWidth(),
                    horizontalArrangement = Arrangement.SpaceBetween,
                    verticalAlignment = Alignment.CenterVertically
                ) {
                    Text(
                        text = if (latencyState.isFallbackToLocalEngineActive) "? Modo
 Local Ativo (VPS Ignorada)" else "? Modo Nuvem Ativo (VPS Preferencial)",
                        color = if (latencyState.isFallbackToLocalEngineActive) RedDec
line else NeonGreen,
                        fontSize = 11.sp,
                        fontWeight = FontWeight.Bold
                    )

                    OutlinedButton(
                        onClick = {
                            val newState = !latencyState.isFallbackToLocalEngineActive
                            HostingerLatencyMonitor.setFallbackActive(newState)
                            val actionLabel = if (newState) "Ativou Fallback Local (<5
ms)" else "Restaurou Conex?o VPS"
                            addLog(
                                source = "GOVERNAN?A REDE",
                                action = "Chaveamento Failover",
                                endpoint = "LOCAL_FALLBACK_SWITCH",
                                status = if (newState) "LOCAL FORCED" else "VPS PRIMAR
Y",
                                latencyMs = 1L,
                                isSuccess = true,
                                details = actionLabel
                            )
                            Toast.makeText(context, actionLabel, Toast.LENGTH_SHORT).s
how()
                        },
                        shape = RoundedCornerShape(8.dp),
                        modifier = Modifier.height(32.dp)
                    ) {
                        Text(if (latencyState.isFallbackToLocalEngineActive) "Restaura
r Nuvem" else "For?ar Fallback Local", fontSize = 10.sp)
                    }
                }
            }
        }

        Spacer(modifier = Modifier.height(12.dp))

        // =======================================================
        // 3. SE??O 2: TESTES DE WEBHOOK ASAAS & SPLIT PIX
        // =======================================================
```

```
            Card(
                shape = RoundedCornerShape(14.dp),
                colors = CardDefaults.cardColors(containerColor = Color(0xFF0F172A)),
                border = androidx.compose.foundation.BorderStroke(1.dp, Color(0xFF334155))
,
                modifier = Modifier.fillMaxWidth()
            ) {
                Column(modifier = Modifier.padding(12.dp)) {
                    Row(
                        modifier = Modifier.fillMaxWidth(),
                        horizontalArrangement = Arrangement.SpaceBetween,
                        verticalAlignment = Alignment.CenterVertically
                    ) {
                        Row(verticalAlignment = Alignment.CenterVertically) {
                            Icon(Icons.Default.AccountBalanceWallet, contentDescription =
null, tint = NeonGreen, modifier = Modifier.size(16.dp))
                            Spacer(modifier = Modifier.width(6.dp))
                            Text("SIMULADOR DE WEBHOOK ASAAS", color = Color.White, fontSi
ze = 12.sp, fontWeight = FontWeight.Bold)
                        }
                        Text(
                            text = "Split 40% L?der / 60% PJ",
                            color = NeonGreen,
                            fontSize = 11.sp,
                            fontWeight = FontWeight.Black
                        )
                    }

                    Spacer(modifier = Modifier.height(6.dp))
                    Text(
                        text = "Assinatura Semanal: R$ 25,00 ? Split: R$ 10,00 (L?der) + R
$ 15,00 (PJ Master)",
                        color = Color(0xFF94A3B8),
                        fontSize = 11.sp
                    )

                    Spacer(modifier = Modifier.height(10.dp))

                    Row(
                        modifier = Modifier.fillMaxWidth(),
                        horizontalArrangement = Arrangement.spacedBy(8.dp)
                    ) {
                        // Bot?o Simular PAYMENT_RECEIVED
                        Button(
                            onClick = {
                                isRunningAsaasWebhook = true
                                coroutineScope.launch {
                                    val simulatedCustomer = "cus_motoboy_${(1000..9999).ra
ndom()}"
                                    val event = AsaasWebhookSimulatorManager.recordWebhook
Event(
                                        context = context,
                                        eventType = "PAYMENT_RECEIVED",
                                        customerId = simulatedCustomer,
                                        customerName = "Thiago Motoboy VIP",
                                        customerPhone = "+55 11 98765-4321",
                                        paymentValue = 25.00,
                                        planType = "Semanal PIX (R$ 25,00) ? Split 40%",
                                        whatsappStatus = "7 Dias PRO Ativados (Split R$ 10
,00 L?der)"
                                    )

                                    // Feedback sonoro e t?til
                                    try {
                                        SaleAlertSoundManager.playSaleNotification(context
)
                                    } catch (_: Exception) {}

                                    try {
                                        HapticFeedbackEngine.vibratePattern(context, Hapti
cPattern.SUCCESS_DOUBLE_PULSE)
                                    } catch (_: Exception) {}

                                    addLog(
                                        source = "ASAAS WEBHOOK",
                                        action = "Pagamento Recebido PIX",
                                        endpoint = "WEBHOOK /webhook/asaas",
                                        status = "200 SUCCESS",
                                        latencyMs = 12L,
                                        isSuccess = true,
                                        details = "Split 40% (R$ 10,00 L?der) + 60% (R$ 15
,00 PJ) -> PRO Liberado 7 dias!"
```

```
                                    )

                                    isRunningAsaasWebhook = false
                                    testResultSummary = "? Webhook Asaas processado: Licen
?a PRO de 7 dias ativada com split R$ 10 L?der / R$ 15 PJ!"
                                    Toast.makeText(context, "Asaas: Licen?a 7 Dias PRO Ati
vada via PIX!", Toast.LENGTH_SHORT).show()
                                }
                            },
                            enabled = !isRunningAsaasWebhook,
                            colors = ButtonDefaults.buttonColors(containerColor = Color(0x
FF059669)),
                            shape = RoundedCornerShape(10.dp),
                            modifier = Modifier
                                .weight(1f)
                                .height(40.dp)
                                .testTag("btn_simulate_payment_received")
                        ) {
                            Icon(Icons.Default.CheckCircle, contentDescription = null, mod
ifier = Modifier.size(16.dp))
                            Spacer(modifier = Modifier.width(4.dp))
                            Text("1. Simular Pago (PRO)", fontSize = 11.sp, fontWeight = F
ontWeight.Bold)
                        }

                        // Bot?o Simular PAYMENT_OVERDUE (Inadimpl?ncia)
                        Button(
                            onClick = {
                                coroutineScope.launch {
                                    val simulatedCustomer = "cus_motoboy_overdue"
                                    AsaasWebhookSimulatorManager.recordWebhookEvent(
                                        context = context,
                                        eventType = "PAYMENT_OVERDUE",
                                        customerId = simulatedCustomer,
                                        customerName = "Condutor Inadimplente",
                                        customerPhone = "+55 11 99999-0000",
                                        paymentValue = 25.00,
                                        planType = "Semanal PIX (R$ 25,00)",
                                        whatsappStatus = "Acesso PRO Bloqueado"
                                    )

                                    addLog(
                                        source = "ASAAS WEBHOOK",
                                        action = "Pagamento Vencido",
                                        endpoint = "WEBHOOK /webhook/asaas",
                                        status = "PAYMENT_OVERDUE",
                                        latencyMs = 8L,
                                        isSuccess = false,
                                        details = "Acesso PRO bloqueado imediatamente. Pla
no revertido para FREE restrito."
                                    )

                                    testResultSummary = "?? Webhook Asaas: Acesso PRO bloq
ueado por vencimento (PAYMENT_OVERDUE)."
                                    Toast.makeText(context, "Asaas: Acesso PRO Bloqueado (
PAYMENT_OVERDUE)", Toast.LENGTH_SHORT).show()
                                }
                            },
                            colors = ButtonDefaults.buttonColors(containerColor = Color(0x
FFDC2626)),
                            shape = RoundedCornerShape(10.dp),
                            modifier = Modifier
                                .weight(1f)
                                .height(40.dp)
                                .testTag("btn_simulate_payment_overdue")
                        ) {
                            Icon(Icons.Default.Warning, contentDescription = null, modifie
r = Modifier.size(16.dp))
                            Spacer(modifier = Modifier.width(4.dp))
                            Text("2. Vencido (Bloqueio)", fontSize = 11.sp, fontWeight = F
ontWeight.Bold)
                        }
                    }

                    Spacer(modifier = Modifier.height(8.dp))

                    // Bot?o Validar Split Matem?tico
                    Row(
                        modifier = Modifier.fillMaxWidth(),
                        horizontalArrangement = Arrangement.SpaceBetween,
                        verticalAlignment = Alignment.CenterVertically
                    ) {
```

```
                        val isSplitValid = AsaasFinancialEngine.validateSplit()
                        Text(
                            text = if (isSplitValid) "? Split Matem?tico 100% Conforme" el
se "? Inconsist?ncia na Regra de Split",
                            color = if (isSplitValid) NeonGreen else RedDecline,
                            fontSize = 11.sp,
                            fontWeight = FontWeight.Bold
                        )

                        OutlinedButton(
                            onClick = {
                                val valid = AsaasFinancialEngine.validateSplit()
                                addLog(
                                    source = "AUDITORIA FISCAL",
                                    action = "Valida??o Split",
                                    endpoint = "ALGORITMO_SPLIT_AUDIT",
                                    status = if (valid) "AUDIT OK" else "AUDIT FAIL",
                                    latencyMs = 0L,
                                    isSuccess = valid,
                                    details = "Total: R$ 25,00 | L?der: R$ 10,00 (40,0%) |
 PJ: R$ 15,00 (60,0%)"
                                )
                                Toast.makeText(context, "Split validado: 40% L?der + 60% P
J = R$ 25,00", Toast.LENGTH_SHORT).show()
                            },
                            shape = RoundedCornerShape(8.dp),
                            modifier = Modifier.height(32.dp)
                        ) {
                            Text("Auditar Split", fontSize = 10.sp)
                        }
                    }
                }
            }

            // Exibi??o do resumo do ?ltimo teste executado
            AnimatedVisibility(visible = testResultSummary != null) {
                testResultSummary?.let { summary ->
                    Box(
                        modifier = Modifier
                            .fillMaxWidth()
                            .padding(vertical = 8.dp)
                            .clip(RoundedCornerShape(10.dp))
                            .background(Color(0xFF0F172A))
                            .border(1.dp, Color(0xFF00D2FF).copy(alpha = 0.5f), RoundedCor
nerShape(10.dp))
                            .padding(10.dp)
                    ) {
                        Text(
                            text = summary,
                            color = Color(0xFFE2E8F0),
                            fontSize = 11.sp,
                            fontWeight = FontWeight.Medium
                        )
                    }
                }
            }

            Spacer(modifier = Modifier.height(12.dp))

            // ========================================================
            // 4. SE??O 3: CONSOLE DE LOGS DE REDE AO VIVO (AUDITORIA)
            // ========================================================
            Row(
                modifier = Modifier.fillMaxWidth(),
                horizontalArrangement = Arrangement.SpaceBetween,
                verticalAlignment = Alignment.CenterVertically
            ) {
                Row(verticalAlignment = Alignment.CenterVertically) {
                    Icon(Icons.Default.Terminal, contentDescription = null, tint = Color(0
xFF00FF88), modifier = Modifier.size(16.dp))
                    Spacer(modifier = Modifier.width(6.dp))
                    Text(
                        text = "CONSOLE DE EVENTOS EM TEMPO REAL (${logs.size})",
                        color = Color.White,
                        fontSize = 12.sp,
                        fontWeight = FontWeight.Bold
                    )
                }

                Row {
                    // Copiar Logs
                    IconButton(
```

```
                        onClick = {
                            if (logs.isNotEmpty()) {
                                val sb = StringBuilder()
                                sb.append("=== LOG DE DIAGN?STICO VPS HOSTINGER & ASAAS ==
=\n")
                                logs.forEach { l ->
                                    sb.append("[${l.timestamp}] [${l.source}] ${l.action}
| ${l.endpointOrEvent} | ${l.status} (${l.latencyMs}ms) -> ${l.details}\n")
                                }
                                val clipboard = context.getSystemService(Context.CLIPBOARD
_SERVICE) as ClipboardManager
                                clipboard.setPrimaryClip(ClipData.newPlainText("Logs Jarvi
s VPS", sb.toString()))
                                Toast.makeText(context, "Logs copiados para a ?rea de tran
sfer?ncia!", Toast.LENGTH_SHORT).show()
                            } else {
                                Toast.makeText(context, "Nenhum log gravado ainda.", Toast
.LENGTH_SHORT).show()
                            }
                        },
                        modifier = Modifier.size(28.dp)
                    ) {
                        Icon(Icons.Default.ContentCopy, contentDescription = "Copiar Logs"
, tint = Color(0xFF94A3B8), modifier = Modifier.size(16.dp))
                    }

                    // Limpar Logs
                    IconButton(
                        onClick = {
                            logs.clear()
                            testResultSummary = null
                            Toast.makeText(context, "Logs limpos com sucesso.", Toast.LENG
TH_SHORT).show()
                        },
                        modifier = Modifier.size(28.dp)
                    ) {
                        Icon(Icons.Default.ClearAll, contentDescription = "Limpar Logs", t
int = Color(0xFF94A3B8), modifier = Modifier.size(16.dp))
                    }
                }
            }

            Spacer(modifier = Modifier.height(6.dp))

            Box(
                modifier = Modifier
                    .fillMaxWidth()
                    .height(180.dp)
                    .clip(RoundedCornerShape(12.dp))
                    .background(Color(0xFF050811))
                    .border(1.dp, Color(0xFF1E293B), RoundedCornerShape(12.dp))
                    .padding(8.dp)
            ) {
                if (logs.isEmpty()) {
                    Box(
                        modifier = Modifier.fillMaxWidth().height(160.dp),
                        contentAlignment = Alignment.Center
                    ) {
                        Text(
                            text = "Aguardando disparo de testes da VPS ou Webhook Asaas..
.\nClique nos bot?es acima para auditar os endpoints em tempo real.",
                            color = Color(0xFF475569),
                            fontSize = 11.sp,
                            textAlign = androidx.compose.ui.text.style.TextAlign.Center,
                            fontFamily = FontFamily.Monospace
                        )
                    }
                } else {
                    LazyColumn(
                        modifier = Modifier.fillMaxWidth(),
                        verticalArrangement = Arrangement.spacedBy(6.dp)
                    ) {
                        items(logs, key = { it.id }) { log ->
                            Column(
                                modifier = Modifier
                                    .fillMaxWidth()
                                    .clip(RoundedCornerShape(6.dp))
                                    .background(Color(0xFF0D1322))
                                    .padding(6.dp)
                            ) {
                                Row(
                                    modifier = Modifier.fillMaxWidth(),
```

```
                                horizontalArrangement = Arrangement.SpaceBetween,
                                verticalAlignment = Alignment.CenterVertically
                            ) {
                                Row(verticalAlignment = Alignment.CenterVertically) {
                                    Box(
                                        modifier = Modifier
                                            .size(6.dp)
                                            .clip(CircleShape)
                                            .background(if (log.isSuccess) NeonGreen e
lse RedDecline)
                                    )
                                    Spacer(modifier = Modifier.width(6.dp))
                                    Text(
                                        text = "[${log.timestamp}] ${log.source}",
                                        color = Color(0xFF00D2FF),
                                        fontSize = 10.sp,
                                        fontWeight = FontWeight.Bold,
                                        fontFamily = FontFamily.Monospace
                                    )
                                }

                                Text(
                                    text = "${log.latencyMs}ms",
                                    color = if (log.latencyMs > 500) RedDecline else N
eonGreen,
                                    fontSize = 10.sp,
                                    fontWeight = FontWeight.Bold,
                                    fontFamily = FontFamily.Monospace
                                )
                            }

                            Spacer(modifier = Modifier.height(2.dp))
                            Text(
                                text = "${log.action} ? ${log.endpointOrEvent} -> ${lo
g.status}",
                                color = Color.White,
                                fontSize = 11.sp,
                                fontWeight = FontWeight.SemiBold,
                                fontFamily = FontFamily.Monospace
                            )

                            Text(
                                text = log.details,
                                color = Color(0xFF94A3B8),
                                fontSize = 10.sp,
                                fontFamily = FontFamily.Monospace
                            )
                        }
                    }
                }
            }
        }
    }
}
```

```
import androidx.compose.material.icons.filled.CheckCircle
import androidx.compose.material.icons.filled.Diamond
import androidx.compose.material.icons.filled.DirectionsBike
import androidx.compose.material.icons.filled.OpenInBrowser
import androidx.compose.material.icons.filled.Route
import androidx.compose.material.icons.filled.Speed
import androidx.compose.material.icons.filled.Splitscreen
import androidx.compose.material.icons.filled.VolumeUp
import androidx.compose.material3.Button
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.HorizontalDivider
import androidx.compose.material3.Icon
import androidx.compose.material3.OutlinedButton
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableIntStateOf
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.platform.testTag
import androidx.compose.ui.text.font.FontFamily
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import java.util.Locale

/**
 * Cen?rio de teste para a simula??o de tela dividida e mesclagem combinat?ria.
 */
data class SplitScenario(
    val id: Int,
    val title: String,
    val subtitle: String,
    val topAppName: String,
    val topPackage: String,
    val topRestaurant: String,
    val topPickup: String,
    val topDropoff: String,
    val topValue: Double,
    val topDistanceKm: Double,
    val bottomAppName: String,
    val bottomPackage: String,
    val bottomRestaurant: String,
    val bottomPickup: String,
    val bottomDropoff: String,
    val bottomValue: Double,
    val bottomDistanceKm: Double,
    val totalRevenue: Double,
    val netProfit: Double,
    val haversineMeters: Int,
    val extraMinutes: Double,
    val alignmentDeg: Float,
    val isApproved: Boolean,
    val badgeLabel: String,
    val badgeColor: Color,
    val reason: String
)

/**
 * DualSplitWindowSimulatorCard
 *
 * Componente visual interativo que simula a tela do celular do motoboy repartida
 * verticalmente em 50% superior (App 1) e 50% inferior (App 2), com HUD flutuante
 * central de ganho combinado, c?lculo geod?sico Haversine e bot?o de divis?o real no Andr
oid.
 */
@Composable
fun DualSplitWindowSimulatorCard(
    modifier: Modifier = Modifier
) {
    val context = LocalContext.current

    val scenarios = remember {
```

```
        listOf(
            SplitScenario(
                id = 1,
                title = "? Cen?rio Diamante (Mesmo Corredor)",
                subtitle = "Av. Paulista -> Consola??o (iFood + 99Food)",
                topAppName = "iFood Entregas",
                topPackage = "com.ifood.entregador",
                topRestaurant = "Madero Container Paulista",
                topPickup = "Av. Paulista, 1200",
                topDropoff = "R. Augusta, 1800",
                topValue = 19.50,
                topDistanceKm = 3.2,
                bottomAppName = "99Food Driver",
                bottomPackage = "com.didiglobal.driver",
                bottomRestaurant = "Outback P?tio Paulista",
                bottomPickup = "R. Treze de Maio, 1947",
                bottomDropoff = "R. Bela Cintra, 1100",
                bottomValue = 21.00,
                bottomDistanceKm = 3.6,
                totalRevenue = 40.50,
                netProfit = 38.20,
                haversineMeters = 280,
                extraMinutes = 1.4,
                alignmentDeg = 12f,
                isApproved = true,
                badgeLabel = "? DESVIO M?NIMO (+280m)",
                badgeColor = Color(0xFF00FF88),
                reason = "As duas coletas distam 450m e entregas distam 280m no mesmo sent
ido vi?rio. Lucro l?quido: R$ 38,20 em 22 min."
            ),
            SplitScenario(
                id = 2,
                title = "? Cen?rio Transversal (Ruas Paralelas)",
                subtitle = "Jardins -> Itaim Bibi (iFood + Rappi)",
                topAppName = "iFood Entregas",
                topPackage = "com.ifood.entregador",
                topRestaurant = "P?o de A??car Express",
                topPickup = "Al. Santos, 980",
                topDropoff = "R. Haddock Lobo, 1300",
                topValue = 16.00,
                topDistanceKm = 2.8,
                bottomAppName = "Rappi Entregador",
                bottomPackage = "com.grability.rappi.entregador",
                bottomRestaurant = "Bullguer Jardins",
                bottomPickup = "R. Bela Cintra, 1900",
                bottomDropoff = "Al. Lorena, 1500",
                bottomValue = 17.50,
                bottomDistanceKm = 3.1,
                totalRevenue = 33.50,
                netProfit = 31.10,
                haversineMeters = 650,
                extraMinutes = 3.2,
                alignmentDeg = 28f,
                isApproved = true,
                badgeLabel = "? DESVIO MODERADO (+650m)",
                badgeColor = Color(0xFFFFB800),
                reason = "Ruas paralelas na mesma regi?o gastron?mica. Desvio aceit?vel de
 3 min extras sem violar o SLA de comida quente."
            ),
            SplitScenario(
                id = 3,
                title = "? Cen?rio de Bloqueio (Contra-fluxo)",
                subtitle = "Santana (Zona Norte) vs Moema (Zona Sul)",
                topAppName = "iFood Entregas",
                topPackage = "com.ifood.entregador",
                topRestaurant = "Habib’s Santana",
                topPickup = "Av. Cruzeiro do Sul, 2000",
                topDropoff = "R. Volunt?rios da P?tria, 1500",
                topValue = 18.00,
                topDistanceKm = 4.0,
                bottomAppName = "99Food Driver",
                bottomPackage = "com.didiglobal.driver",
                bottomRestaurant = "Pizzaria Moema",
                bottomPickup = "Av. Moema, 450",
                bottomDropoff = "Al. dos Maracatins, 800",
                bottomValue = 22.00,
                bottomDistanceKm = 4.5,
                totalRevenue = 40.00,
                netProfit = 22.50,
                haversineMeters = 4850,
                extraMinutes = 24.0,
                alignmentDeg = 175f,
```

```
                isApproved = false,
                badgeLabel = "? REJEITADO PELO MOTOR (-4.8km)",
                badgeColor = Color(0xFFFF5252),
                reason = "Diverg?ncia de rotas de 175? (dire??es opostas). Desvio causaria
 atraso grave e cancelamento pelo cliente."
            )
        )
    }

    var selectedScenarioIndex by remember { mutableIntStateOf(0) }
    val currentScenario = scenarios[selectedScenarioIndex]

    var isSimulatingAcceptance by remember { mutableStateOf(false) }

    Card(
        shape = RoundedCornerShape(22.dp),
        colors = CardDefaults.cardColors(containerColor = DarkCardElevated),
        modifier = modifier
            .fillMaxWidth()
            .padding(vertical = 8.dp)
            .border(1.5.dp, Color(0xFF00D2FF).copy(alpha = 0.6f), RoundedCornerShape(22.dp
))
            .testTag("dual_split_window_simulator_card")
    ) {
        Column(
            modifier = Modifier
                .fillMaxWidth()
                .padding(16.dp)
        ) {
            // ============================================================
            // 1. CABE?ALHO DO SIMULADOR
            // ============================================================
            Row(
                modifier = Modifier.fillMaxWidth(),
                horizontalArrangement = Arrangement.SpaceBetween,
                verticalAlignment = Alignment.CenterVertically
            ) {
                Row(verticalAlignment = Alignment.CenterVertically) {
                    Box(
                        modifier = Modifier
                            .size(36.dp)
                            .clip(CircleShape)
                            .background(Color(0xFF00D2FF).copy(alpha = 0.15f)),
                        contentAlignment = Alignment.Center
                    ) {
                        Icon(Icons.Default.Splitscreen, contentDescription = null, tint =
Color(0xFF00D2FF), modifier = Modifier.size(20.dp))
                    }
                    Spacer(modifier = Modifier.width(10.dp))
                    Column {
                        Text(
                            text = "SIMULADOR DE TELA DIVIDIDA NO GUID?O",
                            color = Color.White,
                            fontSize = 13.sp,
                            fontWeight = FontWeight.Black,
                            letterSpacing = 0.5.sp
                        )
                        Text(
                            text = "50% Topo (App 1) + 50% Base (App 2) + HUD T?tico",
                            color = Color(0xFF00D2FF),
                            fontSize = 11.sp,
                            fontWeight = FontWeight.Medium
                        )
                    }
                }

                Surface(
                    shape = RoundedCornerShape(8.dp),
                    color = currentScenario.badgeColor.copy(alpha = 0.2f),
                    border = androidx.compose.foundation.BorderStroke(1.dp, currentScenari
o.badgeColor)
                ) {
                    Text(
                        text = if (currentScenario.isApproved) "APROVADO" else "BLOQUEADO"
,
                        color = currentScenario.badgeColor,
                        fontSize = 10.sp,
                        fontWeight = FontWeight.Black,
                        modifier = Modifier.padding(horizontal = 6.dp, vertical = 3.dp)
                    )
                }
            }
```

```
            Spacer(modifier = Modifier.height(12.dp))

            // ========================================================
            // 2. SELETOR DE CEN?RIOS PR?TICOS
            // ========================================================
            Text(
                text = "SELECIONE O CEN?RIO DE CORRIDA:",
                color = Color(0xFF94A3B8),
                fontSize = 10.sp,
                fontWeight = FontWeight.Bold,
                letterSpacing = 0.5.sp
            )
            Spacer(modifier = Modifier.height(6.dp))

            Row(
                modifier = Modifier.fillMaxWidth(),
                horizontalArrangement = Arrangement.spacedBy(6.dp)
            ) {
                scenarios.forEachIndexed { idx, sc ->
                    val isSelected = selectedScenarioIndex == idx
                    val btnBorder = if (isSelected) sc.badgeColor else Color(0xFF334155)
                    val btnBg = if (isSelected) sc.badgeColor.copy(alpha = 0.2f) else Colo
r(0xFF0F172A)

                    Surface(
                        shape = RoundedCornerShape(10.dp),
                        color = btnBg,
                        border = androidx.compose.foundation.BorderStroke(1.dp, btnBorder)
,
                        modifier = Modifier
                            .weight(1f)
                            .clickable {
                                selectedScenarioIndex = idx
                                isSimulatingAcceptance = false
                            }
                    ) {
                        Column(
                            modifier = Modifier.padding(vertical = 8.dp, horizontal = 6.dp
),
                            horizontalAlignment = Alignment.CenterHorizontally
                        ) {
                            Text(
                                text = when (idx) {
                                    0 -> "? Diamante"
                                    1 -> "? Transversal"
                                    else -> "? Conflito"
                                },
                                color = if (isSelected) Color.White else Color(0xFF94A3B8)
,
                                fontSize = 11.sp,
                                fontWeight = FontWeight.Bold
                            )
                            Text(
                                text = if (sc.isApproved) "+${sc.haversineMeters}m" else "
Bloqueio",
                                color = sc.badgeColor,
                                fontSize = 10.sp,
                                fontWeight = FontWeight.Black
                            )
                        }
                    }
                }
            }

            Spacer(modifier = Modifier.height(14.dp))

            // ========================================================
            // 3. MOCKUP INTERATIVO DA TELA DO CELULAR DIVIDIDA (50% / 50%)
            // ========================================================
            Box(
                modifier = Modifier
                    .fillMaxWidth()
                    .clip(RoundedCornerShape(18.dp))
                    .background(Color(0xFF030712))
                    .border(2.dp, Color(0xFF1E293B), RoundedCornerShape(18.dp))
                    .padding(8.dp)
            ) {
                Column(modifier = Modifier.fillMaxWidth()) {

                    // ------------------------------------------------
                    // 3.1 METADE SUPERIOR (50%): APP PRIM?RIO (Ex: iFood)
```

```
                    // ------------------------------------------------
                    Surface(
                        shape = RoundedCornerShape(12.dp),
                        color = Color(0xFF180A0A),
                        border = androidx.compose.foundation.BorderStroke(1.2.dp, Color(0x
FFEA1D2C).copy(alpha = 0.7f)),
                        modifier = Modifier.fillMaxWidth()
                    ) {
                        Column(modifier = Modifier.padding(10.dp)) {
                            Row(
                                modifier = Modifier.fillMaxWidth(),
                                horizontalArrangement = Arrangement.SpaceBetween,
                                verticalAlignment = Alignment.CenterVertically
                            ) {
                                Row(verticalAlignment = Alignment.CenterVertically) {
                                    Box(
                                        modifier = Modifier
                                            .size(8.dp)
                                            .clip(CircleShape)
                                            .background(Color(0xFFEA1D2C))
                                    )
                                    Spacer(modifier = Modifier.width(6.dp))
                                    Text(
                                        text = "${currentScenario.topAppName.uppercase()}
(SUPERIOR - 50%)",
                                        color = Color(0xFFEA1D2C),
                                        fontSize = 11.sp,
                                        fontWeight = FontWeight.Black
                                    )
                                }
                                Text(
                                    text = "R$ ${String.format(Locale.US, "%.2f", currentS
cenario.topValue)}",
                                    color = Color(0xFF00FF88),
                                    fontSize = 14.sp,
                                    fontWeight = FontWeight.Black,
                                    fontFamily = FontFamily.Monospace
                                )
                            }

                            Spacer(modifier = Modifier.height(4.dp))
                            Text(
                                text = "? Coleta: ${currentScenario.topRestaurant}",
                                color = Color.White,
                                fontSize = 11.sp,
                                fontWeight = FontWeight.Bold
                            )
                            Text(
                                text = "? Entrega: ${currentScenario.topDropoff} ? ${curre
ntScenario.topDistanceKm} km",
                                color = Color(0xFF94A3B8),
                                fontSize = 10.sp
                            )
                        }
                    }

                    Spacer(modifier = Modifier.height(6.dp))

                    // ------------------------------------------------
                    // 3.2 DIVISOR CENTRAL & FLOATING TACTICAL HUD
                    // ------------------------------------------------
                    Surface(
                        shape = RoundedCornerShape(12.dp),
                        color = Color(0xFF0B132B),
                        border = androidx.compose.foundation.BorderStroke(1.5.dp, currentS
cenario.badgeColor),
                        modifier = Modifier
                            .fillMaxWidth()
                            .padding(vertical = 2.dp)
                    ) {
                        Column(modifier = Modifier.padding(8.dp)) {
                            Row(
                                modifier = Modifier.fillMaxWidth(),
                                horizontalArrangement = Arrangement.SpaceBetween,
                                verticalAlignment = Alignment.CenterVertically
                            ) {
                                Row(verticalAlignment = Alignment.CenterVertically) {
                                    Icon(
                                        imageVector = if (currentScenario.isApproved) Icon
s.Default.Diamond else Icons.Default.Block,
                                        contentDescription = null,
                                        tint = currentScenario.badgeColor,
```

```
                                    modifier = Modifier.size(16.dp)
                                )
                                Spacer(modifier = Modifier.width(6.dp))
                                Text(
                                    text = "HUD CENTRAL: R$ ${String.format(Locale.US,
 "%.2f", currentScenario.totalRevenue)}",
                                    color = Color.White,
                                    fontSize = 12.sp,
                                    fontWeight = FontWeight.Black
                                )
                            }

                            Text(
                                text = "Lucro L?q: R$ ${String.format(Locale.US, "%.2f
", currentScenario.netProfit)}",
                                color = Color(0xFF00FF88),
                                fontSize = 11.sp,
                                fontWeight = FontWeight.Black,
                                fontFamily = FontFamily.Monospace
                            )
                        }

                        Spacer(modifier = Modifier.height(3.dp))

                        Row(
                            modifier = Modifier.fillMaxWidth(),
                            horizontalArrangement = Arrangement.SpaceBetween
                        ) {
                            Text(
                                text = "Desvio Haversine: +${currentScenario.haversine
Meters}m (+${currentScenario.extraMinutes} min)",
                                color = currentScenario.badgeColor,
                                fontSize = 10.sp,
                                fontWeight = FontWeight.Bold
                            )

                            Text(
                                text = "Azimute: ${currentScenario.alignmentDeg.toInt(
)}?",
                                color = Color(0xFF00D2FF),
                                fontSize = 10.sp,
                                fontWeight = FontWeight.Bold
                            )
                        }
                    }
                }

                Spacer(modifier = Modifier.height(6.dp))

                // ---------------------------------------------------
                // 3.3 METADE INFERIOR (50%): APP SECUND?RIO (Ex: 99Food)
                // ---------------------------------------------------
                Surface(
                    shape = RoundedCornerShape(12.dp),
                    color = Color(0xFF1A1608),
                    border = androidx.compose.foundation.BorderStroke(1.2.dp, Color(0x
FFFFB800).copy(alpha = 0.7f)),
                    modifier = Modifier.fillMaxWidth()
                ) {
                    Column(modifier = Modifier.padding(10.dp)) {
                        Row(
                            modifier = Modifier.fillMaxWidth(),
                            horizontalArrangement = Arrangement.SpaceBetween,
                            verticalAlignment = Alignment.CenterVertically
                        ) {
                            Row(verticalAlignment = Alignment.CenterVertically) {
                                Box(
                                    modifier = Modifier
                                        .size(8.dp)
                                        .clip(CircleShape)
                                        .background(Color(0xFFFFB800))
                                )
                                Spacer(modifier = Modifier.width(6.dp))
                                Text(
                                    text = "${currentScenario.bottomAppName.uppercase(
)} (INFERIOR - 50%)",
                                    color = Color(0xFFFFB800),
                                    fontSize = 11.sp,
                                    fontWeight = FontWeight.Black
                                )
                            }
                            Text(
```

```
                                    text = "R$ ${String.format(Locale.US, "%.2f", currentS
cenario.bottomValue)}",
                                    color = Color(0xFF00FF88),
                                    fontSize = 14.sp,
                                    fontWeight = FontWeight.Black,
                                    fontFamily = FontFamily.Monospace
                                )
                            }

                            Spacer(modifier = Modifier.height(4.dp))
                            Text(
                                text = "? Coleta: ${currentScenario.bottomRestaurant}",
                                color = Color.White,
                                fontSize = 11.sp,
                                fontWeight = FontWeight.Bold
                            )
                            Text(
                                text = "? Entrega: ${currentScenario.bottomDropoff} ? ${cu
rrentScenario.bottomDistanceKm} km",
                                color = Color(0xFF94A3B8),
                                fontSize = 10.sp
                            )
                        }
                    }
                }
            }

            Spacer(modifier = Modifier.height(10.dp))

            // ========================================================
            // 4. ITINER?RIO CONSOLIDADO DE 4 PASSOS
            // ========================================================
            Card(
                shape = RoundedCornerShape(12.dp),
                colors = CardDefaults.cardColors(containerColor = Color(0xFF0B101D)),
                border = androidx.compose.foundation.BorderStroke(1.dp, Color(0xFF1E293B))
,
                modifier = Modifier.fillMaxWidth()
            ) {
                Column(modifier = Modifier.padding(10.dp)) {
                    Text(
                        text = "ROTA DE EXECU??O OTIMIZADA PELO MOTOR:",
                        color = Color(0xFF64748B),
                        fontSize = 10.sp,
                        fontWeight = FontWeight.Bold
                    )
                    Spacer(modifier = Modifier.height(4.dp))

                    if (currentScenario.isApproved) {
                        Text("1. Coletar no ${currentScenario.topRestaurant} (${currentSce
nario.topAppName})", color = Color(0xFFE2E8F0), fontSize = 11.sp)
                        Text("2. Coletar no ${currentScenario.bottomRestaurant} (${current
Scenario.bottomAppName}) - Desvio: +180m", color = Color(0xFFE2E8F0), fontSize = 11.sp)
                        Text("3. Entregar em ${currentScenario.topDropoff}", color = Color
(0xFFE2E8F0), fontSize = 11.sp)
                        Text("4. Entregar em ${currentScenario.bottomDropoff} - Desvio fin
al: +${currentScenario.haversineMeters}m", color = Color(0xFFE2E8F0), fontSize = 11.sp)
                    } else {
                        Text(
                            text = "? CORRIDA BLOQUEADA: ${currentScenario.reason}",
                            color = Color(0xFFFF5252),
                            fontSize = 11.sp,
                            fontWeight = FontWeight.Bold
                        )
                    }
                }
            }

            Spacer(modifier = Modifier.height(14.dp))

            // ========================================================
            // 5. BOT?ES DE ACIONAMENTO REAL
            // ========================================================
            Row(
                modifier = Modifier.fillMaxWidth(),
                horizontalArrangement = Arrangement.spacedBy(8.dp)
            ) {
                // Bot?o 1: Acionar Tela Dividida Real no Android
                Button(
                    onClick = {
                        val coordinator = SplitScreenDualRadarCoordinator.getInstance(cont
ext)
```

```
                        val pair = SplitScreenAppPair(
                            id = "pair_sim_${currentScenario.id}",
                            title = "${currentScenario.topAppName} + ${currentScenario.bot
tomAppName}",
                            topPackage = currentScenario.topPackage,
                            topAppName = currentScenario.topAppName,
                            bottomPackage = currentScenario.bottomPackage,
                            bottomAppName = currentScenario.bottomAppName,
                            description = "Par ativado via simulador no guid?o"
                        )

                        try {
                            HapticFeedbackHelper.vibrateSuccess(context)
                            coordinator.launchSplitScreenPair(context, pair)
                            Toast.makeText(context, "? Dividindo tela: ${currentScenario.t
opAppName} (topo) e ${currentScenario.bottomAppName} (base)", Toast.LENGTH_SHORT).show()
                        } catch (e: Exception) {
                            Toast.makeText(context, "Disparando altern?ncia de tela dividi
da...", Toast.LENGTH_SHORT).show()
                        }
                    },
                    colors = ButtonDefaults.buttonColors(containerColor = Color(0xFF0284C7
)),
                    shape = RoundedCornerShape(12.dp),
                    modifier = Modifier
                        .weight(1f)
                        .height(44.dp)
                        .testTag("btn_trigger_real_split")
                ) {
                    Icon(Icons.Default.Splitscreen, contentDescription = null, modifier =
Modifier.size(18.dp))
                    Spacer(modifier = Modifier.width(6.dp))
                    Text("Dividir Tela Real", fontSize = 11.sp, fontWeight = FontWeight.Bo
ld)
                }

                // Bot?o 2: Simular Aceite Simult?neo e Alerta de Voz
                Button(
                    onClick = {
                        isSimulatingAcceptance = true
                        try {
                            HapticFeedbackHelper.vibrateSuccess(context)
                            val voiceText = if (currentScenario.isApproved) {
                                "Combo aprovado de R$ ${String.format(Locale.US, "%.2f", c
urrentScenario.totalRevenue)}. Coleta um no ${currentScenario.topRestaurant}. Desvio de ap
enas ${currentScenario.haversineMeters} metros."
                            } else {
                                "Aten??o: Corrida descartada por desvio excessivo no senti
do oposto."
                            }
                            TextToSpeechManager.getInstance(context).speakVoiceAlert(voice
Text)
                            Toast.makeText(context, "? $voiceText", Toast.LENGTH_LONG).sho
w()
                        } catch (_: Exception) {}
                    },
                    colors = ButtonDefaults.buttonColors(
                        containerColor = if (currentScenario.isApproved) Color(0xFF059669)
 else Color(0xFFDC2626)
                    ),
                    shape = RoundedCornerShape(12.dp),
                    modifier = Modifier
                        .weight(1f)
                        .height(44.dp)
                        .testTag("btn_simulate_accept_combo")
                ) {
                    Icon(
                        imageVector = if (currentScenario.isApproved) Icons.Default.CheckC
ircle else Icons.Default.Block,
                        contentDescription = null,
                        modifier = Modifier.size(18.dp)
                    )
                    Spacer(modifier = Modifier.width(6.dp))
                    Text(if (currentScenario.isApproved) "Simular Aceite" else "Testar Blo
queio", fontSize = 11.sp, fontWeight = FontWeight.Bold)
                }
            }
        }
    }
}
```

```
import android.content.Context
import android.util.Log
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import org.json.JSONArray
import org.json.JSONObject

/**
 * Par?metros de Regra de Neg?cio do Motor Financeiro Asaas (Passo 1):
 * 1. Assinatura Semanal: R$ 25,00 via PIX Recorrente
 * 2. Split Autom?tico:
 *    - 40% (R$ 10,00) fixo para a conta Asaas do L?der de Pra?a (walletId do General)
 *    - 60% (R$ 15,00) l?quido retido na Conta Master PJ do App
 * 3. Webhooks:
 *    - PAYMENT_RECEIVED: Ativa licen?a PRO por 7 dias
 *    - PAYMENT_OVERDUE: Bloqueia acesso PRO no app imediatamente
 */
object AsaasFinancialEngine {

    private const val TAG = "AsaasFinancialEngine"

    // Par?metros Oficiais de Precifica??o e Split
    const val WEEKLY_SUBSCRIPTION_PRICE = 25.00 // R$ 25,00
    const val LEADER_SPLIT_PERCENTAGE = 40.0    // 40%
    const val LEADER_FIXED_VALUE = 10.00        // R$ 10,00 fixo
    const val MASTER_PJ_PERCENTAGE = 60.0       // 60%
    const val MASTER_PJ_FIXED_VALUE = 15.00     // R$ 15,00 fixo
    const val CYCLE_TYPE = "WEEKLY"
    const val BILLING_TYPE = "PIX"

    // Wallet ID padr?o do General / L?der de Pra?a (configur?vel por pra?a/regi?o)
    const val DEFAULT_GENERAL_WALLET_ID = "wal_general_praca_sp_01"

    private val _engineState = MutableStateFlow(
        AsaasEngineState(
            subscriptionPrice = WEEKLY_SUBSCRIPTION_PRICE,
            leaderSplitValue = LEADER_FIXED_VALUE,
            masterPjSplitValue = MASTER_PJ_FIXED_VALUE,
            generalWalletId = DEFAULT_GENERAL_WALLET_ID,
            isSplitValid = true
        )
    )
    val engineState: StateFlow<AsaasEngineState> = _engineState.asStateFlow()

    data class AsaasEngineState(
        val subscriptionPrice: Double,
        val leaderSplitValue: Double,
        val masterPjSplitValue: Double,
        val generalWalletId: String,
        val isSplitValid: Boolean,
        val lastWebhookProcessed: String? = null
    )

    /**
     * Valida matematicamente a conformidade da regra de split.
     */
    fun validateSplit(
        totalPrice: Double = WEEKLY_SUBSCRIPTION_PRICE,
        leaderValue: Double = LEADER_FIXED_VALUE,
        masterValue: Double = MASTER_PJ_FIXED_VALUE
    ): Boolean {
        val sum = leaderValue + masterValue
        val isSumEqual = Math.abs(sum - totalPrice) < 0.001
        val leaderRatio = leaderValue / totalPrice
        val masterRatio = masterValue / totalPrice
        val isLeader40 = Math.abs(leaderRatio - 0.40) < 0.001
        val isMaster60 = Math.abs(masterRatio - 0.60) < 0.001
        return isSumEqual && isLeader40 && isMaster60
    }

    /**
     * Gera o Payload JSON oficial de cria??o de Assinatura Semanal com Split no Asaas API
 v3.
     */
    fun buildAsaasSubscriptionPayload(
        customerAsaasId: String,
        customerName: String,
        generalWalletId: String = _engineState.value.generalWalletId,
        nextDueDate: String // Formato YYYY-MM-DD
    ): JSONObject {
```

```
        val payload = JSONObject()
        payload.put("customer", customerAsaasId)
        payload.put("billingType", BILLING_TYPE)
        payload.put("value", WEEKLY_SUBSCRIPTION_PRICE)
        payload.put("nextDueDate", nextDueDate)
        payload.put("cycle", CYCLE_TYPE)
        payload.put("description", "Assinatura Semanal JARVIS NEURAL COCKPIT PRO (PIX Reco
rrente)")

        // Configura??o de Split Autom?tico conforme documenta??o oficial do Asaas
        val splitArray = JSONArray()
        val leaderSplit = JSONObject()
        leaderSplit.put("walletId", generalWalletId)
        leaderSplit.put("fixedValue", LEADER_FIXED_VALUE) // R$ 10,00 fixo (40%)
        leaderSplit.put("description", "Comiss?o L?der de Pra?a (40%)")
        splitArray.put(leaderSplit)

        payload.put("split", splitArray)
        return payload
    }

    /**
     * Processa evento de Webhook recebido do Asaas na VPS ou simulador.
     * Executa a regra de neg?cio do PASSO 1:
     * - PAYMENT_RECEIVED: Ativa licen?a por 7 dias no SubscriptionManager
     * - PAYMENT_OVERDUE: Bloqueia acesso PRO no app imediatamente
     */
    fun processWebhookEvent(
        context: Context,
        eventType: String,
        customerId: String,
        value: Double
    ): Boolean {
        Log.i(TAG, "Processando Webhook Asaas: $eventType | Cliente: $customerId | Valor:
R$ $value")

        return when (eventType) {
            "PAYMENT_RECEIVED", "PAYMENT_CONFIRMED" -> {
                // Ativa licen?a de 7 dias (Assinatura Semanal)
                SubscriptionManager.initialize(context)
                SubscriptionManager.activateWeeklySubscription()
                _engineState.value = _engineState.value.copy(
                    lastWebhookProcessed = "PAYMENT_RECEIVED: Licen?a 7 dias ATIVA para $c
ustomerId"
                )
                true
            }

            "PAYMENT_OVERDUE" -> {
                // Bloqueia acesso PRO imediatamente (volta ao plano FREE restrito)
                SubscriptionManager.initialize(context)
                SubscriptionManager.resetToFree()
                _engineState.value = _engineState.value.copy(
                    lastWebhookProcessed = "PAYMENT_OVERDUE: Acesso PRO BLOQUEADO para $cu
stomerId"
                )
                true
            }

            else -> {
                Log.d(TAG, "Evento informativo Asaas n?o-bloqueante: $eventType")
                false
            }
        }
    }
}
```

```
 */
object NavigationQuickDispatcher {

    private const val TAG = "NavQuickDispatcher"

    /**
     * Inicia navega??o GPS imediata no Waze
     */
    fun launchWaze(
        context: Context,
        latitude: Double,
        longitude: Double,
        addressTitle: String = ""
    ): Boolean {
        return try {
            val uri = Uri.parse("waze://?ll=$latitude,$longitude&navigate=yes")
            val intent = Intent(Intent.ACTION_VIEW, uri).apply {
                addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
                setPackage("com.waze")
            }

            if (intent.resolveActivity(context.packageManager) != null) {
                context.startActivity(intent)
                announceNavStart(context, "Waze", addressTitle)
                true
            } else {
                // Fallback: abre via link web ou Google Maps
                launchGoogleMaps(context, latitude, longitude, addressTitle)
            }
        } catch (e: Exception) {
            Log.e(TAG, "Falha ao abrir Waze: ${e.message}", e)
            launchGoogleMaps(context, latitude, longitude, addressTitle)
        }
    }

    /**
     * Inicia navega??o GPS no Google Maps
     */
    fun launchGoogleMaps(
        context: Context,
        latitude: Double,
        longitude: Double,
        addressTitle: String = ""
    ): Boolean {
        return try {
            // Intent otimizado de navega??o turn-by-turn do Google Maps
            val gmmIntentUri = Uri.parse("google.navigation:q=$latitude,$longitude&mode=d"
)
            val mapIntent = Intent(Intent.ACTION_VIEW, gmmIntentUri).apply {
                addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
                setPackage("com.google.android.apps.maps")
            }

            if (mapIntent.resolveActivity(context.packageManager) != null) {
                context.startActivity(mapIntent)
                announceNavStart(context, "Google Maps", addressTitle)
                true
            } else {
                // Fallback gen?rico geo:
                val geoUri = Uri.parse("geo:$latitude,$longitude?q=${Uri.encode(addressTit
le.ifBlank { "$latitude,$longitude" })}")
                val fallbackIntent = Intent(Intent.ACTION_VIEW, geoUri).apply {
                    addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
                }
                context.startActivity(fallbackIntent)
                announceNavStart(context, "Mapa GPS", addressTitle)
                true
            }
        } catch (e: Exception) {
            Log.e(TAG, "Falha ao abrir Google Maps: ${e.message}", e)
            Toast.makeText(context, "Nenhum aplicativo de GPS encontrado.", Toast.LENGTH_S
HORT).show()
            false
        }
    }

    /**
     * Dispara o melhor navegador baseado na prefer?ncia ou disponibilidade
     */
    fun launchBestRoute(
        context: Context,
        latitude: Double,
```

```
        longitude: Double,
        addressTitle: String = "",
        preferWaze: Boolean = true
    ) {
        if (preferWaze) {
            launchWaze(context, latitude, longitude, addressTitle)
        } else {
            launchGoogleMaps(context, latitude, longitude, addressTitle)
        }
    }

    /**
     * Avisa o piloto via TTS que a rota foi recalculada ap?s cancelamento de sub-pedido e
m corrida mesclada.
     */
    fun announceRouteRecalculated(context: Context, remainingDestination: String, newKm: D
ouble) {
        try {
            HapticFeedbackHelper.vibrateAccept(context)
            val kmText = String.format(Locale.ROOT, "%.1f", newKm)
            val tts = TextToSpeechManager.getInstance(context)
            tts.speak("Rota recalculada: nova entrega para $remainingDestination, restando
 $kmText quil?metros.", priority = true)
        } catch (_: Exception) {}
    }

    /**
     * Avisa o piloto via TTS que a corrida mesclada foi abortada e retorna para espera de
 chamadas.
     */
    fun announceRouteAborted(context: Context) {
        try {
            HapticFeedbackHelper.vibrateDecline(context)
            val tts = TextToSpeechManager.getInstance(context)
            tts.speak("Corrida abortada com sucesso. Retornando ao radar de espera.", prio
rity = true)
        } catch (_: Exception) {}
    }

    private fun announceNavStart(context: Context, appName: String, address: String) {
        try {
            HapticFeedbackHelper.vibrateSuccess(context)
            val destText = if (address.isNotBlank()) " para $address" else ""
            val tts = OfferTextToSpeechEngine.getInstance(context)
            tts.speak("Iniciando rota no $appName$destText.")
        } catch (_: Exception) {}
    }
}
```

```
for superior a 10 km/h.
 * 4. Desbloquear a interface automaticamente quando o ve?culo parar ou reduzir abaixo do
limiar (com histerese anti-oscila??o).
 * 5. Notificar ouvintes (TTS, UI de Bloqueio, HUD Flutuante) das mudan?as de estado da tr
ava.
 */
class SpeedSafetyLockUtil private constructor(private val context: Context) {

    companion object {
        private const val TAG = "SpeedSafetyLockUtil"

        // Limiar obrigat?rio de seguran?a: 10.0 km/h
        const val LOCK_SPEED_THRESHOLD_KMH = 10.0

        // Limiar inferior de histerese para desbloquear (evita oscila??o na borda de 10 k
m/h)
        const val UNLOCK_HYSTERESIS_THRESHOLD_KMH = 8.5

        @Volatile
        private var instance: SpeedSafetyLockUtil? = null

        fun getInstance(context: Context): SpeedSafetyLockUtil {
            return instance ?: synchronized(this) {
                instance ?: SpeedSafetyLockUtil(context.applicationContext).also { instanc
e = it }
            }
        }
    }

    /**
     * Modelo de estado do utilit?rio de seguran?a
     */
    data class State(
        val currentSpeedKmh: Double = 0.0,
        val isLocked: Boolean = false,
        val thresholdKmh: Double = LOCK_SPEED_THRESHOLD_KMH,
        val latitude: Double = -23.561684,
        val longitude: Double = -46.655981,
        val accuracyMeters: Float = 0f,
        val isTrackingActive: Boolean = false,
        val isSimulationMode: Boolean = false,
        val statusMessage: String = "Monitoramento inativo"
    )

    private val fusedLocationClient: FusedLocationProviderClient =
        LocationServices.getFusedLocationProviderClient(context)

    private var locationCallback: LocationCallback? = null

    private val _state = MutableStateFlow(State())
    val state: StateFlow<State> = _state.asStateFlow()

    private var lastLocation: Location? = null
    private var lastLocationTimestamp: Long = 0L

    var onLockStateChanged: ((isLocked: Boolean, speedKmh: Double) -> Unit)? = null

    /**
     * Inicia o monitoramento de velocidade em tempo real via FusedLocationProviderClient
     */
    @SuppressLint("MissingPermission")
    fun startTracking(): Boolean {
        if (!hasLocationPermission()) {
            Log.w(TAG, "Permiss?o de localiza??o n?o concedida para o SpeedSafetyLockUtil.
")
            _state.value = _state.value.copy(
                isTrackingActive = false,
                statusMessage = "Aguardando permiss?o de GPS"
            )
            return false
        }

        if (_state.value.isTrackingActive) {
            return true
        }

        stopTracking()

        val locationRequest = LocationRequest.Builder(Priority.PRIORITY_HIGH_ACCURACY, 100
0L).apply {
            setMinUpdateIntervalMillis(500L)
            setMinUpdateDistanceMeters(0.5f)
```

```
            setWaitForAccurateLocation(false)
        }.build()

        locationCallback = object : LocationCallback() {
            override fun onLocationResult(result: LocationResult) {
                val loc = result.lastLocation ?: return
                processNewLocation(loc)
            }
        }

        try {
            fusedLocationClient.requestLocationUpdates(
                locationRequest,
                locationCallback!!,
                Looper.getMainLooper()
            )
            _state.value = _state.value.copy(
                isTrackingActive = true,
                statusMessage = "GPS Fused Location ativo (Limite: 10 km/h)"
            )
            Log.i(TAG, "Monitoramento FusedLocationProviderClient iniciado com sucesso.")
            return true
        } catch (e: Exception) {
            Log.e(TAG, "Erro ao registrar atualiza??es de localiza??o: ${e.message}", e)
            _state.value = _state.value.copy(
                isTrackingActive = false,
                statusMessage = "Falha ao iniciar GPS: ${e.message}"
            )
            return false
        }
    }

    /**
     * Interrompe o monitoramento de localiza??o do GPS
     */
    fun stopTracking() {
        locationCallback?.let {
            try {
                fusedLocationClient.removeLocationUpdates(it)
            } catch (e: Exception) {
                Log.w(TAG, "Erro ao remover atualiza??es de localiza??o: ${e.message}")
            }
        }
        locationCallback = null
        _state.value = _state.value.copy(isTrackingActive = false)
    }

    /**
     * Processa a leitura do GPS, computa a velocidade real e aplica a trava de seguran?a
     */
    private fun processNewLocation(location: Location) {
        if (_state.value.isSimulationMode) return

        var calculatedSpeedKmh = 0.0

        // 1. Tenta usar o hardware speed do GPS (m/s -> km/h)
        if (location.hasSpeed() && location.speed >= 0f) {
            calculatedSpeedKmh = location.speed.toDouble() * 3.6
        } else {
            // 2. Fallback de c?lculo geod?sico (delta dist?ncia / delta tempo)
            val prev = lastLocation
            val now = System.currentTimeMillis()
            if (prev != null && lastLocationTimestamp > 0L) {
                val dtSeconds = (now - lastLocationTimestamp) / 1000.0
                if (dtSeconds in 0.5..10.0) {
                    val distMeters = prev.distanceTo(location)
                    val speedMps = distMeters / dtSeconds
                    calculatedSpeedKmh = (speedMps * 3.6).coerceIn(0.0, 180.0)
                }
            }
        }

        lastLocation = location
        lastLocationTimestamp = System.currentTimeMillis()

        updateSpeedAndEvaluateLock(
            speedKmh = calculatedSpeedKmh,
            lat = location.latitude,
            lng = location.longitude,
            accuracy = location.accuracy
        )
    }
```

```
    /**
     * Aplica a l?gica da trava com histerese (trava > 10 km/h e destrava < 8.5 km/h)
     */
    private fun updateSpeedAndEvaluateLock(
        speedKmh: Double,
        lat: Double,
        lng: Double,
        accuracy: Float
    ) {
        val currentLocked = _state.value.isLocked
        val newLocked = when {
            speedKmh >= LOCK_SPEED_THRESHOLD_KMH -> true
            speedKmh <= UNLOCK_HYSTERESIS_THRESHOLD_KMH -> false
            else -> currentLocked // Faixa de histerese para evitar oscila??o
        }

        val wasLocked = currentLocked
        val speedRounded = Math.round(speedKmh * 10.0) / 10.0

        _state.value = _state.value.copy(
            currentSpeedKmh = speedRounded,
            isLocked = newLocked,
            latitude = lat,
            longitude = lng,
            accuracyMeters = accuracy,
            statusMessage = if (newLocked) {
                "? TRAVA ATIVA (> 10 km/h) ? Interface Bloqueada para Seguran?a"
            } else {
                "? Interface Liberada (Velocidade Segura)"
            }
        )

        // Se o estado mudou, notifica os ouvintes
        if (wasLocked != newLocked) {
            Log.w(TAG, "Estado da trava de seguran?a alterado: $wasLocked -> $newLocked (V
elocidade: $speedRounded km/h)")
            onLockStateChanged?.invoke(newLocked, speedRounded)

            // Feedback sonoro e t?tico
            if (newLocked) {
                try {
                    TextToSpeechManager.getInstance(context).speak(
                        "Aten??o: velocidade acima de 10 por hora detectada por GPS. Trava
 de seguran?a ativada. N?o use a tela em movimento."
                    )
                } catch (_: Exception) {}
            } else {
                try {
                    TextToSpeechManager.getInstance(context).speak(
                        "Ve?culo parado ou abaixo do limite. Interface liberada."
                    )
                } catch (_: Exception) {}
            }
        }
    }

    /**
     * Simula velocidade para testes r?pidos em bancada sem necessidade de moto em movimen
to
     */
    fun simulateSpeed(simSpeedKmh: Double) {
        _state.value = _state.value.copy(isSimulationMode = true)
        updateSpeedAndEvaluateLock(
            speedKmh = simSpeedKmh,
            lat = _state.value.latitude,
            lng = _state.value.longitude,
            accuracy = 2.5f
        )
    }

    /**
     * Desativa o modo de simula??o e retorna ao GPS real do FusedLocationProviderClient
     */
    fun resetSimulation() {
        _state.value = _state.value.copy(isSimulationMode = false)
        startTracking()
    }

    private fun hasLocationPermission(): Boolean {
        val fine = ContextCompat.checkSelfPermission(
            context,
```

```
            android.Manifest.permission.ACCESS_FINE_LOCATION
        ) == PackageManager.PERMISSION_GRANTED
        val coarse = ContextCompat.checkSelfPermission(
            context,
            android.Manifest.permission.ACCESS_COARSE_LOCATION
        ) == PackageManager.PERMISSION_GRANTED
        return fine || coarse
    }
}
```